



Domain Counterfactual Data Augmentation for Explainable Recommendation

YI YU, Kyoto University, Kyoto, Japan

KAZUNARI SUGIYAMA, Osaka Seikei University, Osaka, Japan

ADAM JATOWT, University of Innsbruck, Innsbruck, Austria

Providing explanations for recommendation decisions is crucial for enhancing user trust and satisfaction in recommender systems. However, existing generative methods often produce generic, repetitive explanation texts that fail to reflect the true reasons behind user interests and item attributes. Thus, it is important to address this *degeneration* issue in recommendation explanations. This work tackles a key problem in explainable recommendation: understanding how *explanation degeneration occurs* and improving explanation quality by mitigating it. We argue that examining the causal mechanism underlying the data generation process is key to addressing this problem. Along this line, we identify a neglected hidden variable, which we refer to as *textual attributes*. Textual attributes encompass various aspects, such as text style, word frequency distributions, and more. Just like user persona and item attributes in traditional recommender systems, textual attributes also shape the nature of explanations. Our analysis of the causal graph reveals the underlying cause of the model's degeneration. To address this issue, we propose a novel learning method called *Domain for Counterfactual Reasoning* (D4C). By using the auxiliary domain to generate counterfactual data and combining it with factual data, this approach helps the model focus more on the causal contributions of users and items during training. Extensive experiments on five real-world datasets from various platforms demonstrate the effectiveness of our approach.

CCS Concepts: • **Information systems** → *Recommender systems*; • **Computing methodologies** → **Natural language generation**;

Additional Key Words and Phrases: Explainable recommender system, natural language generation, counterfactual data augmentation

ACM Reference format:

Yi Yu, Kazunari Sugiyama, and Adam Jatowt. 2025. Domain Counterfactual Data Augmentation for Explainable Recommendation. *ACM Trans. Inf. Syst.* 43, 3, Article 58 (February 2025), 30 pages.
<https://doi.org/10.1145/3711856>

1 Introduction

In the era of digital commerce and content streaming services, recommender systems play a pivotal role in guiding user choices by suggesting items that align with individual preferences. While traditional recommender systems excel in predicting and recommending items based on user-item

Authors' Contact Information: Yi Yu (corresponding author), Kyoto University, Kyoto, Japan; e-mail: yu.yi.87s@st.kyoto-u.ac.jp; Kazunari Sugiyama, Osaka Seikei University, Osaka, Japan; e-mail: sugiyama-k@g.osaka-seikei.ac.jp; Adam Jatowt, University of Innsbruck, Innsbruck, Austria; e-mail: adam.jatowt@uibk.ac.at.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1558-2868/2025/2-ART58

<https://doi.org/10.1145/3711856>

interactions, they often fall short in explaining the rationale behind these recommendations. This gap hinders user trust and satisfaction, as users seek not only personalized recommendations but also understandable justifications for why certain items are recommended [50, 76]. To bridge this gap, the field of explainable recommendation has emerged, focusing on providing personalized explanations to justify recommendation decisions [6, 8, 28, 34, 70, 78]. These explanations aim to enhance user satisfaction, trust, and engagement by elucidating the underlying reasons for suggesting specific items.

Deep generative models have achieved significant success in recent years, leading to remarkable advancements in explainable recommender systems and drawing considerable attention from the research community [3, 7, 24–26, 44, 71, 74]. While these methods differ in their specific approaches and implementations, they share the core concept of layering an explanation model on top of traditional recommendation algorithms. We abstract these methods into a unified workflow, illustrated in Figure 1. The explanation model, which can be implemented using either an encoder-decoder architecture [14] or a decoder-only architecture [25], takes the output of the recommendation model as input, maps it on a semantic feature space, and employs auto-regressive techniques to generate natural language explanations. Since the explanation model integrates both user and item information, the generated explanations of systems can be naturally viewed as outcomes of justifying user–item interactions. It is worth noting that in most works on explainable recommendation, a common practice is to de-emphasize the focus on the recommendation algorithm in the workflow, typically opting for simpler models like neural collaborative filtering [18] or multi-layer perceptrons. This allows for a better focus on the explanation algorithm. We adhere to this practice in our work.

Recent studies [7, 26, 74] find that these generation models often generate repetitive texts during inference, which deviates from the goal of personalization. This phenomenon, known as the *degeneration issue* in explanations, presents a significant challenge [33, 73]. For instance, Li et al. [24, 25] proposed personalized decoder architectures for text generation to guide the language modeling process. Sun et al. [45] introduced dual-form alignment constraints, where explanations are utilized for predicting ratings, and ratings are then used to generate explanations. In [16, 26], **pre-trained language models (PLMs)** with retrieval-augmented generation and prompt learning have been used to leverage the larger models and more data to tackle the degeneration problem. Other methods like [7, 13, 74] design decoders to integrate additional knowledge such as textual aspects or images from reviews, to enrich the model’s context during both training and inference. However, the causes of degeneration remain largely unexplored, limiting the potential for improvement. Unlike existing works that focus on designing network architectures, we argue that the failure to consider inherent data patterns is the key to degeneration and needs to be explicitly modeled. We draw ideas from recent studies on large language models, which show that repetition in data is a major factor causing degeneration in neural text generation [22, 63]. Inspired by these findings, we use structural causal graphs [40], which naturally represent the data generation process, to interpret how model degeneration occurs in recommender systems.

Figure 2(a) illustrates that existing methods assume explanatory data comes purely from the interaction between users and items: U (user node) and I (item node) are the causes, and E (explanation node) is the result.¹ Under this assumption, a notable example is the generative models in [24, 25, 27], which use recurrent neural networks or Transformer-based architectures to “translate” user and item latent factors into concise sentences expressing a user’s experience with an item as an explanation. However, this assumption has a major problem: it ignores various confounding factors

¹Note that, we are focusing on the data generation process in the real world, where users on e-commerce platforms write reviews for items and we then use those reviews as source of explanations to be generated, same as in related works [24, 37].

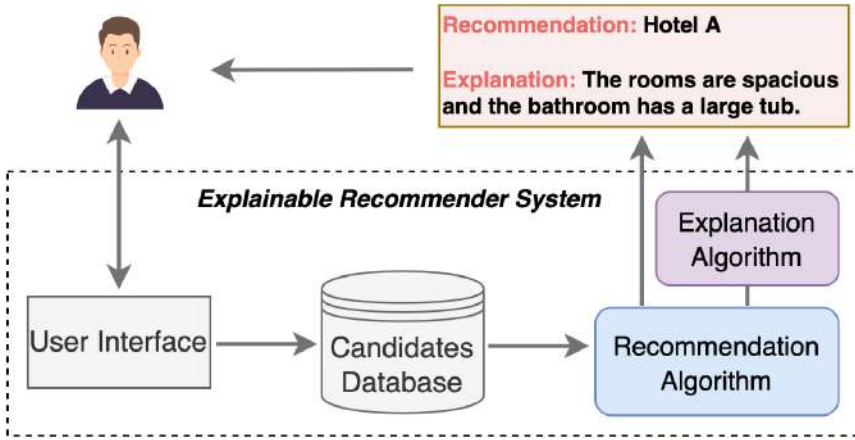


Fig. 1. The simplified workflow of an explainable recommender system, illustrated using an example from the TripAdvisor dataset [24].

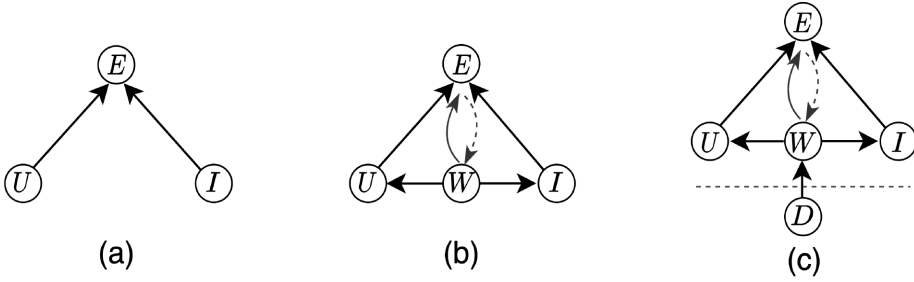


Fig. 2. The causal graphs depict the explanatory data generation process. U represents user node, I is item node, W is text node, D is domain node, and E is explanation node. See the main text for more details. (a) shows that existing methods assume explanatory data comes solely from user preferences and item properties, treating these entities as independent and ignoring various contextual factors. (b) presents our extended causal graph, which incorporates contextual factors influencing both user preferences and item perception, with text attributes (W) serving as a concrete semantic representation of the context. (c) further incorporates a domain variable (D) that determines different environments, enabling us to consider different scenarios that reflect the variations in contextual factors when generating explanations. See the main text in Section 1 for more details.

that may affect user preferences in practical scenarios. In real-world recommender systems, entities such as users and items are not independent but are influenced by complex social or contextual factors [68]. For instance, on e-commerce platforms, users are often influenced by the content of others' reviews when deciding whether to buy an item, while on streaming platforms, users often read comments from other viewers to shape their perceptions on the contents. Hence, ignoring these confounding factors makes it sub-optimal to design models based on the assumptions encoded in Figure 2(a).

To account for these contextual factors, we introduce a variable W in Figure 2(b), which denotes the text attributes of the current environment. Text attributes can provide a concrete semantic representation of the environment, as information flows on e-commerce or streaming platforms are primarily mediated through text. These text attributes can be reflected by the aggregation of various types of texts (e.g., user reviews, product descriptions, comments), representing domain-specific themes (e.g., electronic products) or platform-specific characteristics (e.g., price-friendly).

We briefly explain how these elements interact: The contextual factor W may affect both user preferences ($W \rightarrow U$) and item perception ($W \rightarrow I$) while also influencing the explanations ($\{U, I, W\} \rightarrow E$). The rationality of the causal graph can be found in Section 4.1. Based on our constructed causal graph, we can interpret how model degeneration occurs. Suppose we design a model that uses U and I to estimate E , i.e., $P(E|U, I)$ and optimize it through **maximum likelihood estimation (MLE)**. The problem arises in the model's learning process: since associative learning captures all dependencies among variables [42], including W , the significant predictive signals from W for E lead the model to rely on shortcut dependencies during training through $U \leftarrow W \rightarrow E$ or $I \leftarrow W \rightarrow E$ (see Section 4). In consequence, the model becomes overconfident in assuming it has acquired the necessary knowledge, even though it has not sufficiently learned users' preferences and items' characteristics through the edges $U \rightarrow E \leftarrow I$. Without imposing constraints on those edges, the model fails to capture the full interaction dynamics, making it difficult to generalize to unseen user-item pairs and resulting in generating generic uninformative explanations.

Through the above analysis, we understand that the key to addressing degeneration is to encourage the model to focus on genuine user-item interactions when generating explanations, rather than relying on shortcuts through contextual text attributes. To this end, we propose using causality as a form of regularization to guide the model in learning direct interactions by paths $U \rightarrow E \leftarrow I$. The challenge lies in the fact that we cannot directly observe the contextual factors influencing user-item interactions, and therefore, the values of the hidden variable W remain unavailable. Our idea is shown in Figure 2(c), where we introduce the domain variable D , which affects the system by determining the distribution of text attributes W . By applying counterfactual reasoning on D and treating W as a mediator, we pose the counterfactual query: *What would the explanation be if the user-item interaction had been under different text attributes environment (counterfactual world) instead of the current one (factual world)?* This enables us to disentangle the interactions from the original environmental factors and simulate a different context to generate counterfactual training samples. By combining these counterfactual samples with factual data, we account for the variability in contextual factors when generating explanations, thereby controlling their influence. In this way, the model achieves learning user-item interactions independently of the effects of contextual text attributes. The main contributions of this work are summarized as follows:²

- We provide an intuitive understanding of degeneration from a causal perspective. To the best of our knowledge, this is the first work to integrate causal reasoning into natural language explainable recommendation.
- We propose a novel counterfactual framework that leverages auxiliary domains to generate domain counterfactual training samples, effectively mitigating the degeneration issue.
- Extensive experiments conducted on five real-world datasets validate the effectiveness of our proposed approach.

The remainder of this article is organized as follows: We discuss the related work in Section 2. In Section 3, we introduce the notations and prerequisite knowledge necessary to help readers gain a better understanding of the article. We analyze the degeneration issue and present our method in Section 4. In Section 5, we experiment on real-world datasets to verify the effectiveness of our approach and make discussions. Finally, we conclude this work and discuss future directions in Section 6.

²The code is available at: <https://github.com/yiyualt/D4C>

2 Related Work

2.1 Explainable Recommender System

Section 1 gives a detailed discussion of past studies closely aligned with our work. In this section, we provide a broader perspective on the evolution of explainable recommender systems and position our contribution within this landscape. Explainability has long been a crucial aspect of recommender systems. Initially, efforts were made toward learning interpretable user and item feature representations through text analysis of reviews. For instance, McAuley and Leskovec [36] introduced a matrix factorization approach where latent parameters are designed to grow monotonically over time, improving both accuracy and interpretability of recommendations. They extended this concept in [35] by integrating latent factors with topic modeling, yielding feature representations that are both meaningful and interpretable. Building on these ideas, the field has seen significant advancements through sophisticated models, such as RMR [31], JMARS [10], LMLF [1], FLAME [61], RBLT [48], and MMALFM [8].

As deep learning and attention mechanisms have become prevalent in recommender systems, the method of emphasizing the most influential text fragments, such as words or phrases, emerged as a mainstream technique in explainable recommendation. For instance, attention scores calculated by models predict user preferences and visually highlight significant parts of the inputs, like review texts, through heat maps. This method enables pinpointing the most influential elements in making predictions. Notable models based on this concept include EFM [78], MTER [55], NARRE [4], MPCN [49], AARM [15], DAML [32], AHN [11], and R3 [38]. Unlike methods that learn interpretable feature representations for users/items, this approach can provide explanations specific to each interaction. Consequently, the explanations for why a particular user prefers different products can vary. However, this type of methods faces challenges related to the influence of noisy review texts on attention scores, and the extent to which these attentive explanations accurately reflect the underlying reasons for recommendations has not been thoroughly investigated.

Recently, leveraging generative models to generate natural language explanations has become a preferred method in explainable recommender systems. Due to their powerful text generation capabilities and the natural human interaction offered by these models, they are becoming increasingly popular in explainable recommendation works. The idea is to utilize user characteristics and item attributes to guide the generative model (e.g., an auto-regressive model) in generating sentences that explain why the user likes a particular item. Key models in this line include Ref2Seq [37], NETE [24], DualPC [45], PETER [25], U-ARM [46], SAER [71], ReXPlug [16], PEVAE [3], METER [13], PEPLER [26], and ERRA [7]. Our work is aligned with this line of research. A major challenge is that models often produce repetitive or generic explanations, limiting their ability to provide personalized insights for different users and items. Previous approaches, such as dual-form alignment [45], controlled generation [25], data retrieval augmentation [7, 16], and prompt-based training [26], have been used to address this issue. To the best of our knowledge, our work is the first to unveil the root of degeneration from a causal perspective. For a more detailed discussion on the differences between this work and previous studies, refer to Section 1.

2.2 Counterfactual Reasoning in Recommender Systems

Counterfactual thinking centers on the inquiry, “What would have happened if,” which involves altering the input of data to explore potential outputs. The samples generated are referred to as *counterfactual samples*, or simply *counterfactuals* [42, 52]. Since counterfactual samples are useful for the analysis and training of models, considerable research has focused on the automated generation of high-quality counterfactual data. In recommender systems, for example, Wang et al. [59] devised both data- and model-oriented approaches for generating counterfactual training sequences for

sequential recommendation. The core idea is to replace an item in the training sequence with another to maximize the loss on the target item. Xiong et al. [62] proposed creating counterfactual samples by modifying aspect scores in user feature representations, whereas Tan et al. [47] altered aspect scores in item feature representations to achieve variations in recommendation rankings. It is a natural idea to use the generated counterfactual samples for data augmentation to better train models. Since the inputs of the counterfactual data are similar to the original data but yield different outputs, this method enhances the model's ability to discriminate between minor changes, thereby increasing the model's robustness, as demonstrated in [19, 54, 58, 59, 62, 72]. Moreover, counterfactual reasoning has been increasingly applied in recommender systems to improve model robustness, mitigate biases, and enhance interpretability [56, 60, 64, 67]. For instance, Wei et al. [60] employed counterfactual reasoning to remove the direct effect of item popularity on ranking scores, thereby mitigating popularity bias in recommender systems. Similarly, Wang et al. [56] addressed the clickbait issue by performing counterfactual inference via estimating click likelihood in an imaginary world, where items only have exposure features. Xu et al. [64] integrated counterfactual reasoning into collaborative filtering to address Simpson's paradox and improve recommendation performance. Xu et al. [67] identified echo chambers using causal graph with cycles in recommender systems and apply counterfactual reasoning with backdoor adjustment to solve it. Unlike these works, we address the degeneration issue in explainable recommender systems, filling the gap in causal inference for natural language explainable recommendations. For additional background on causal inference and counterfactual reasoning in recommendations, we recommend readers refer to [5, 12, 40, 43, 64, 66, 67, 69, 77, 79].

3 Preliminaries

3.1 Notation

Before introducing our approach, it is important to clarify the notations used throughout this article. We employ calligraphic fonts to denote sets and spaces: $\mathcal{U}$, $\mathcal{I}$, $\mathcal{S}$, and $\mathcal{E}$ represent the user set, item set, space of rating scores, and space of explanation sentences, respectively. Subscripts indicate the data index: $\mathcal{S}_{u,i}$ and $\mathcal{E}_{u,i}$ denote the ground-truth ratings and explanations given by user u to item i , respectively. $\hat{\mathcal{S}}_{u,i}$ and $\hat{\mathcal{E}}_{u,i}$ represent the model's predictions for these elements. The training set is denoted by $\mathcal{X}$. Superscripts denote domain labels: a and t refer to the auxiliary and target domains, respectively. $\mathcal{X}^a$ and $\mathcal{X}^t$ represent the auxiliary and target domain training data, respectively. Boldface symbols are used to indicate vectors and matrices. Key notations and descriptions are presented in Table 1.

3.2 Problem Formulation

Definition 1 (Explainable Recommendation Task). Let $\mathcal{U}$ and $\mathcal{I}$ denote the sets of users and items, respectively. Each user $u \in \mathcal{U}$ and each item $i \in \mathcal{I}$ are characterized by feature vectors that capture their respective attributes. The explainable recommendation task is defined as follows:

- *Inputs:* A user $u \in \mathcal{U}$ and an item $i \in \mathcal{I}$.
- *Outputs:*
 - *Predicted Rating Score* $\hat{\mathcal{S}}_{u,i}$: A numerical score estimating the suitability of item i for user u . This score is generated by the recommendation model based on the representations of both u and i .
 - *Generated Explanation* $\hat{\mathcal{E}}_{u,i}$: A natural language text that provides a rationale for recommending item i to user u , justifying the recommendation by reflecting the interaction between their representations.

Table 1. Notations

Symbol	Description
$\mathcal{U}, \mathcal{I}, \mathcal{S}, \mathcal{E}$	Sets of users, items, rating scores, and explanation sentences, respectively.
$u, i, \mathcal{S}_{u,i}, \mathcal{E}_{u,i}$	A user u and an item i , with their respective ground-truth ratings and explanations.
$\hat{\mathcal{S}}_{u,i}, \hat{\mathcal{E}}_{u,i}$	Predicted rating and explanation for user u and item i .
$\mathcal{X}, \mathcal{X}^a, \mathcal{X}^t$	Training data, auxiliary domain training data, and target domain training data, respectively.
$\mathbf{f}_u, \mathbf{f}_i$	Feature representations (embeddings) of user u and item i , respectively.
$\mathbf{h}$	Hidden state vectors.
$\mathbf{W}, \mathbf{b}$	Weight matrix and bias vector, respectively.
Θ	Model parameters to be learned.
l	Length of the explanation sentence.
C	Vocabulary of words used in explanations.
U, I, D, W, E	Variables in the causal graph representing user, item, domain, text attributes, and explanation, respectively.

The objective of the explainable recommendation task is not only to accurately predict the preference score ($\hat{\mathcal{S}}_{u,i}$) but also to generate a coherent and meaningful explanation ($\hat{\mathcal{E}}_{u,i}$) that enhances user trust and satisfaction. Given that our primary focus is on the explanation generation aspect rather than the recommendation algorithm itself, we only focus on the explanation generation model. Refer to [45, 71] for more comprehensive details on recommendation techniques.

The explanation generation model, or *generator*, is defined as a function $G : \mathcal{U} \times \mathcal{I} \rightarrow \mathcal{E}$, which maps a user-item pair on an explanation sentence. Specifically, for each user u and item i , the generator function is expressed as $G(u, i) \rightarrow \mathcal{E}_{u,i}$, where $\mathcal{E}_{u,i}$ is a sequence of words $[e_1, e_2, \dots, e_l]$, with l representing the length of the sequence. The process of generating $\mathcal{E}_{u,i}$ is formulated as a conditional language modeling problem, where the generation of each word in the sequence is conditioned on the representations of both the user u and the item i .

The probability of generating the explanation $\mathcal{E}_{u,i}$ is defined as the product of the probabilities of each word e_t in the sequence, conditioned on preceding words and the user-item interaction:

$$P(\mathcal{E}_{u,i}) = P(e_1, e_2, \dots, e_l) = P(e_1|u, i) \cdot P(e_2|e_1; u, i) \cdots P(e_l|e_1, \dots, e_{l-1}; u, i) = \prod_{t=1}^l P(e_t|e_{<t}; u, i). \quad (1)$$

At each timestep t , the hidden state $\mathbf{h}_t \in \mathbb{R}^k$ is computed using the previous hidden state $\mathbf{h}_{t-1}$ and the representations of the user and the item $\mathbf{f}_u, \mathbf{f}_i$, respectively. This is achieved through a neural network g , defined as

$$\mathbf{h}_t = g(\mathbf{h}_{t-1}, \mathbf{f}_u, \mathbf{f}_i). \quad (2)$$

The probability distribution for the word e_t is then obtained by applying a Softmax function to the logits derived from the hidden state $\mathbf{h}_t$, mapping the logits into a probability distribution over the vocabulary C :

$$P(e_t|e_{<t}; u, i) = \text{Softmax}(\mathbf{W} \cdot \mathbf{h}_t + \mathbf{b}), \quad (3)$$

where $\mathbf{W} \in \mathbb{R}^{|C| \times k}$ is the transformation matrix, and $\mathbf{b} \in \mathbb{R}^{|C|}$ is the bias vector. Here, k denotes the dimension of the feature representations.

The generator G is trained to maximize the log likelihood of generating the ground-truth explanations for all user–item interactions in the training set $\mathcal{X}$:

$$\max_G \mathbb{E}_{(u,i,\mathcal{E}_{u,i}) \sim \mathbb{P}(u,i,\mathcal{E}_{u,i})} [\log P(\mathcal{E}_{u,i}|u,i;\Theta)] = \frac{1}{|\mathcal{X}|} \sum_{(u,i,\mathcal{E}_{u,i}) \in \mathcal{X}} \log P(\mathcal{E}_{u,i}|u,i;\Theta), \quad (4)$$

where Θ denotes the learnable parameters of the generator G .

The function g can be implemented using either an recurrent neural network or Transformer architecture [51]. During training, the function employs the teacher forcing technique, where the ground-truth words from the target sequence guide the prediction of subsequent words. During inference, it generates text auto-regressively using its previously produced words as input to predict the next words.

4 Methodology

4.1 Motivation: How Degeneration Comes into Being

Degeneration of explanation is the tendency of models to generate generic or repetitive explanations, failing to reflect individual user or item characteristics [25, 33, 63]. In this section, we use the proposed causal graphs to analyze the causes of degeneration. For ease of comprehension, we present the causal graph in Figure 3 and explain it as follows:

- The nodes U, I, E, D, W denote the user, item, explanation, domain, and text attributes, respectively. U and I denote their IDs, while E is a sequence of words that forms the explanation. D is a domain identifier that signifies different domains. In this work, we focus on two domains, where a is the auxiliary domain and t is the target domain. W denotes the text attributes in the current environment, which is a set of hidden variables. It condenses core topics, keywords, and styles, reflecting domain-specific themes (e.g., plots or actors in movies, performance or price in electronics) and platform-specific nuances (e.g., travel-focused restaurant discussion on TripAdvisor vs. savoring-focused on Yelp). Text attributes can serve as a concrete semantic representation of the environment, as information flows on e-commerce or streaming platforms are largely mediated through texts.
- The edges $\{U, I, W\} \rightarrow E$ indicate that the explanation is influenced by a combination of the user’s interests, the item’s specific features, and the contextual text attributes. The focus of a user’s explanation for an item may vary with context. For a restaurant, these attributes may give rise to savoring-focused explanations if the context is characterized by flavor and taste. On the other hand, if they highlight travel-focused aspects like convenience, the explanation related to the restaurant may focus on these aspects instead. The text attributes reflect such focuses.
- The edge $E \rightarrow W$ indicates that the user-written explanations become part of the recommender system’s textual contents and hence contribute to the system’s text attributes over time.
- The edge $W \rightarrow U$ denotes that contextual text attributes may affect user preferences. Especially, users without strong initial preferences often conform to the majority view and are influenced by highly emphasized aspects. For example, if text attributes in the environment emphasize “affordability” and “ease of use,” users may view these as key features of items in the platform and adjust their expectations and focus accordingly.
- The edge $W \rightarrow I$ denotes that contextual text attributes affect how items are perceived, influencing their visibility and appeal to users. For instance, on a platform where characteristics highlight convenience and travel-friendliness, restaurants with features “close to metro” or “quick meals” may attract more attention.
- The edge $D \rightarrow W$ indicates that domain identifiers lead to different systems with distinct text attributes or characteristics. Specifically, the change of D determines different W , representing

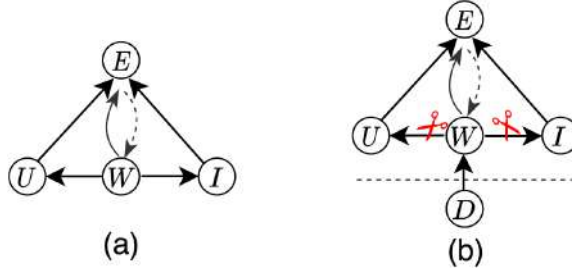


Fig. 3. Causal graphs for explanation generation. U , user node; I , item node; W , text node; D , domain node; E , explanation node.

platform-specific focuses (e.g., travel-focused restaurant discussions on TripAdvisor vs. those on Yelp) or domain-specific themes (e.g., plots or actors in movies, soundtrack or lyrics in music). A complete example is shown in Section 4.3.

As the causal graphs contain the cycle $E \leftrightarrow W$, making causal reasoning complex, we follow [30, 67] to unfold them into acyclic graphs while preserving the dynamic properties using a superscript t . As such, at each timestep t , the graphs are simplified into a directed acyclic form, and we represent the feedback edge with a dashed line.

We start by examining the connection between the contextual text attributes W and explanations E based on the data generation process. As shown in Figure 3(a), at any timestep t , the explanation $E^{(t)}$, conditioned on user U and item I , can be expressed as follows (using marginalization and the chain rule):

$$P(E^{(t)}|U, I) = \sum_{W^{(t)}} P(E^{(t)}, W^{(t)}|U, I) = \sum_{W^{(t)}} P(E^{(t)}|U, I, W^{(t)})P(W^{(t)}|U, I). \quad (5)$$

This equation shows that the explanations $P(E^{(t)}|U, I)$ inherently includes the influence of the current contextual text attributes W , as reflected in the terms $P(E^{(t)}|U, I, W^{(t)})$ and $P(W^{(t)}|U, I)$, reflecting intrinsic data generation process.

Once $E^{(t)}$ is generated, it in turn influences the text attributes $W^{(t+1)}$, as indicated by the edge $E \rightarrow W$. The value of $W^{(t+1)}$ is thus updated by a combination of its previous state and the generated explanation $E^{(t)}$ as follows:

$$P(W^{(t+1)}|U, I) = \alpha \cdot P(E^{(t)}|W^{(t)}, U, I) + (1 - \alpha) \cdot P(W^{(t)}|U, I), \quad (6)$$

where $0 < \alpha < 1$ denotes the extent to which the generated explanations influence the update of text attributes.

Now, moving forward to the next timestep $t + 1$, the generation of $E^{(t+1)}$ also depends on the updated text attributes $W^{(t+1)}$. This can be expressed as

$$P(E^{(t+1)}|U, I) = \sum_{W^{(t+1)}} P(E^{(t+1)}|U, I, W^{(t+1)})P(W^{(t+1)}|U, I). \quad (7)$$

Then, at the subsequent timestep $t + 2$, we can expand this recursive pattern as follows:

$$\begin{aligned} P(W^{(t+2)}|U, I) &= \alpha \cdot P(E^{(t+1)}|U, I, W^{(t+1)}) + (1 - \alpha) \cdot P(W^{(t+1)}|U, I) \\ &= \alpha \cdot P(E^{(t+1)}|U, I, W^{(t+1)}) + \alpha(1 - \alpha) \cdot P(E^{(t)}|U, I, W^{(t)}) + (1 - \alpha)^2 \cdot P(W^{(t)}|U, I). \end{aligned} \quad (8)$$

This formulation shows that the influence of E continuously shapes W , while the impact of the original text attributes $W^{(t)}$ gradually diminishes.

If t keeps increasing to t' , the distributions of $W^{(t')}$ can be expressed by substituting the expression for each timestep as follows:

$$P(W^{(t')}|U, I) = \alpha \sum_{k=0}^{t'} (1 - \alpha)^k P(E^{(t'-k-1)}|U, I, W^{(t'-k-1)}) + (1 - \alpha)^{t'} P(W^{(0)}|U, I). \quad (9)$$

From this equation, it is clear that in case of t' is large and α is large, the latter $(1 - \alpha)^{t'} P(W^{(0)}|U, I)$ becomes vanished, leading to the approximation:

$$P(W^{(t')}|U, I) \approx \alpha \sum_{k=0}^{t'} (1 - \alpha)^k P(E^{(t'-k-1)}|U, I, W^{(t'-k-1)}). \quad (10)$$

This final equation means that, as time progresses, the distribution $P(W^{(t')}|U, I)$ converges toward patterns shaped by the generation of explanations E . This convergence is reasonable: since W represents text-related features, it is natural that they may end up being shaped by the characteristics of the content provided by users. This, in turn, justifies our definition of W as contextual text attributes. It is important to clarify that t here does not refer to a timestep in a sequential model but rather to real-world time. In practice, recommender systems' designers usually collect data for model training only after the system has accumulated a significant number of interactions. Consequently, by the time we design and train the model, t is already quite large, indicating that contextual text attributes have been well established. In the above, we explained the causal graph from the perspective of data generation.

Now, we examine how it impacts the model's prediction process. Suppose we use traditional learning methods, as in [24, 25], with a sequential model generator G_Θ to generate explanations: $\mathcal{E} = G_\Theta(U, I) \propto P(Y|U, I)$, optimized using MLE (Equation (4)). During training, as associative learning captures all dependencies among variables, the model learns not only the interactional relation $U \rightarrow E \leftarrow I$ from the U and I nodes but also information through W to E via the edges $U \leftarrow W \rightarrow E$ and $I \leftarrow W \rightarrow E$. However, as we have shown, W is primarily shaped by E , which means it is a highly predictive signal for E . As a result, the generator may become overconfident by relying on the shortcut through W by learning $P(W|UI)$. The generator assumes it has learned the necessary knowledge, even though it has not adequately learned the interaction edges $U \rightarrow E \leftarrow I$. Consequently, without proper constraints on those edges, the model is prone to overfitting and fails to capture their features and genuine interactions during optimization. This causes the model to struggle with unseen user-item pairs and leads to degeneration of explanations during inference.

4.2 Theoretical Analysis on Mitigating Degeneration

The core of our approach is to optimize the model using the conventional associative learning objective while also optimizing the causal impact of users and items in generating explanations. The first objective ensures the model generates coherent sentences, while the second encourages the model not to dependent on shortcuts during learning. Based on the analysis in Section 4.1, we see that degeneration occurs when the model heavily relies on learning shortcuts during training, which undermines its ability to learn the causal edges. Therefore, our second learning objective is directly designed to address the degeneration. In this section, we focus on this.

To represent the causal effect of user-item interactions on generating explanations, we utilize the *do*-operator from causal inference [41, 42]. For example, $P(E|do(U, I))$ denotes the probability of explanations when user-item interactions are intervened and independent of other variables. Since in real-world systems, users and items are affected by other variables, it is common practice to estimate it from observational data. In some scenarios, it is possible to collect data that satisfy the backdoor criterion [75, 77] or the front-door criterion [68], making it possible to estimate causal

effects directly from observational data. However, in our case, the text attributes W are latent and cannot be directly observed or measured. To address this challenge, we employ counterfactual reasoning for causal learning [64, 67].

As shown in Figure 3(b), we introduce the domain node D , which influences the system by changing the distribution of the text attributes W . By incorporating knowledge from auxiliary domains through cross-domain learning frameworks [9, 53, 74], we are able to construct counterfactual scenarios to address the question: *What would the explanation be if the user-item interaction had occurred in a different domain with different text attributes?* This process generates what we refer to as *domain counterfactuals*—hypothetical samples that signifies the potential explanations under different contextual W values. By incorporating counterfactual samples alongside factual data, we account for the variations in W during explanation generation, effectively controlling its influence [77]. In this way, it enables the generator to learn user-item interaction independently of the influence of W . To formalize this, we first introduce Pearl’s *do*-calculus rules and then present our approach in mathematical forms.

THEOREM 1 (PEARL’S DO-CALCULUS [41, 42]). *Let $P(Y|do(X))$ be the effect of an intervention $do(X = x)$ on a variable Y , with other variables W, Z , and V , where the relationships among variables are encoded in a graph $\mathcal{G}$. The *do*-calculus provides the following three rules:*

- *Rule 1 (Insertion/Deletion of Observations): If Y and Z are d -separated by X in $\mathcal{G}_{\overline{X}}$ (the graph where edges into X are removed), then we have: $P(Y|do(X), Z, W) = P(Y|do(X), W)$.*
- *Rule 2 (Action/Observation Exchange): If Y and X are d -separated by Z in $\mathcal{G}_{\underline{X}}$ (the graph where edges from X are removed), then we have: $P(Y|do(X), Z, W) = P(Y|X, Z, W)$.*
- *Rule 3 (Insertion/Deletion of Actions): If Y and X are d -separated by Z in $\mathcal{G}_{\overline{X}, \underline{Z}}$ (the graph where edges into X and outgoing edges from Z are removed), then we have: $P(Y|do(X), do(Z), W) = P(Y|do(Z), W)$.*

Note that the above rules are presented in a generalized form. We can simplify these equations by eliminating common symbols from both sides. For instance: *Rule 1:* $P(Y|do(X), Z) = P(Y|do(X))$, *Rule 2:* $P(Y|do(X), Z) = P(Y|X, Z)$, *Rule 3:* $P(Y|do(X), do(Z)) = P(Y|do(Z))$, with the conditions for holding equality being adjusted accordingly.

do-calculus helps formalize and simplify the expression for causal effects, that is, expressing $P(Y|do(X))$ in terms of statistical expressions, often used in conjunction with causal graphs [12, 42, 57, 65]. Based on Figure 3(b) and using *do*-calculus rules, we can derive the causal effect of users and items on explanations, i.e., $P(E|do(U, I))$ as follows:

$$\begin{aligned}
 P(E|do(U, I)) &\stackrel{(1)}{=} \sum_D P(E, D|do(U, I)) \quad (\text{marginalization}) \\
 &\stackrel{(2)}{=} \sum_D P(E|do(U, I), D) \cdot P(D|do(U, I)) \quad (\text{chain rule}) \\
 &\stackrel{(3)}{=} \sum_D P(E|do(U, I), D) \cdot P(D) \quad (\text{do-calculus rule 3}) \\
 &\stackrel{(4)}{=} \sum_w \sum_D P(E, w|do(U, I), D) \cdot P(D) \quad (\text{marginalization}) \\
 &\stackrel{(5)}{=} \sum_w \sum_D P(E|do(U, I), D, w) \cdot P(D) \cdot P(w|do(U, I), D) \quad (\text{chain rule})
 \end{aligned}$$

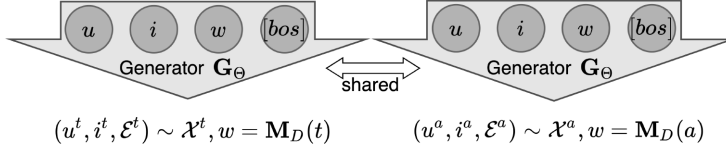
$$\begin{aligned}
&\stackrel{(6)}{=} \sum_w \sum_D P(E|U, I, D, w) \cdot P(D) \cdot P(w|do(U, I), D) \quad (\text{do-calculus rule 2}) \\
&\stackrel{(7)}{=} \sum_w \sum_D P(E|U, I, w) \cdot P(D) \cdot P(w|do(U, I), D) \quad (\text{do-calculus rule 1}) \\
&\stackrel{(8)}{=} \sum_w \sum_D P(E|U, I, w) \cdot P(D) \cdot P(w|D) \quad (\text{do-calculus rule 3}) \\
&\stackrel{(9)}{=} \frac{1}{2} \sum_D \sum_w P(E|U, I, w) \cdot P(w|D) \quad (\text{replace } P(D)) \\
&\stackrel{(10)}{=} \frac{1}{2} (\mathbb{E}_{w \sim P(w|D=t)} [P(E|U, I, w)] + \mathbb{E}_{w \sim P(w|D=a)} [P(E|U, I, w)]) \quad (\text{expectation form}). \quad (11)
\end{aligned}$$

In this derivation, we start by marginalizing over the domain variable D that affects W , followed by marginalizing over W that affects U and I , and with multiple *do*-calculus rules. At step (9), we set $P(D) = \frac{1}{2}$ because in our setup we use only one target domain and one auxiliary domain. In the final step (10), we expand the equation for both domains and express the result as an expectation. The final expression reveals that the causal effect of user–item interactions on explanations is the average of the probabilities of explanations generated under different text attributes across domains. Importantly, this requires combining both the factual world (where $D = t$) and counterfactual world (where $D = a$), meaning that we need to apply counterfactual reasoning to generate counterfactual training samples. In practice, our goal is not to compute the exact values of the causal effects, but to disentangle them from spurious correlations in model learning. When we combine these counterfactual samples with factual data, we account for the variability in W when generating explanations, thereby controlling for its influences. As a result, the model is optimized to capture the disentangled causal relations of user–item interactions on explanations while avoiding overfitting to specific text patterns. We will detail the model optimization process in Section 4.4.

4.3 Counterfactual Reasoning

Counterfactual reasoning allows for more refined intervention at the individual level [42, 64, 67]. Our goal of counterfactual reasoning is to generate counterfactual training samples, so as to mitigate degeneration according to Equation (11). To this, we follow a three-step procedure [42]: (i) *Abduction*: Using observed data as evidence to determine the posterior of hidden variables for fixing each individual. In recommender systems, where user and item are represented as deterministic embeddings, this step is equivalent to infer the embedding vectors based on the observed data. (ii) *Action*: For each user–item in target domain, we replace the contextual text attributes w with alternative ones $\tilde{w}$ from the auxiliary domain. (iii) *Prediction*: We infer the counterfactual explanation $\tilde{E}$ under the changed text attributes $\tilde{w}$ for each user–item in target domain. Figure 4 illustrates an overview of the counterfactual generation process. We provide an intuitive example to demonstrate this process in the below.

Suppose that there is a user, Tom, who is a big fan of the movie *Titanic*. In the target domain, where the attribute w is related to “movie”-specific features such as actors, performances, and plot, Tom’s explanation would focus on *Titanic*’s movie characters, like Rose and Jack’s iconic sunset scene. In another domain, where w is related to “music”-specific features, such as the original soundtrack, lyrics, and melody, Tom’s explanation might shift to comments about how great *Titanic*’s theme song is. In the abduction step, the model first infers that Tom’s preference is on *Titanic*. Then, in the action step, the contextual text attributes w of “movie” are replaced with “music.” Finally, in the prediction step, the model generates possible explanation related to *Titanic*’s music. This example also explains why we need to introduce an auxiliary domain—to

1. Abduction**2. Action**

$$(u^t, i^t, \mathcal{E}^t) \sim \mathcal{X}^t, w = \mathbf{M}_D(t) \rightarrow \tilde{w} = \mathbf{M}_D(a)$$

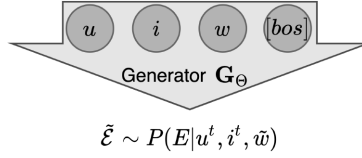
3. Prediction

Fig. 4. The three-step procedure of counterfactual generation. Refer to Section 4.3 for detailed explanations.

simulate such domain-specific variations. We use this example to recall the degeneration problem: During the model's optimization process (i.e., when learning $P(E|U, I)$), W can act as a shortcut. This implies that the model may heavily rely on learning contextual text attributes such as narrative textual patterns describing plots, box office performance, and actors, which could result in an overestimation of its understanding on capturing the user's interest in *Titanic* and insufficiently learning of users' interests, thus leading to model degeneration. In the following, we provide a detailed explanation of each step.

Abduction. The goal of abduction step is to estimate the values of the variables in the causal graph shown in Figure 3 based on the observed evidence. Specifically, we work with target domain data, $\mathcal{X}^t = \{u^t, i^t, \mathcal{E}^t\}$ and auxiliary domain data, $\mathcal{X}^a = \{u^a, i^a, \mathcal{E}^a\}$, corresponding to the value of $D = t$ and $D = a$, respectively. Additionally, we need to infer the values of W in the causal graph. Since W represents hidden domain-specific features, we can utilize a domain embedding matrix $\mathbf{M}_D \in \mathbb{R}^{2 \times k}$, to initialize their values. That is, for the target domain, $w = \mathbf{M}_D(t) \in \mathbb{R}^k$, and for the auxiliary domain, $w = \mathbf{M}_D(a) \in \mathbb{R}^k$. In this way, it allows us to achieve the probabilistic process $w \sim P(w|D)$ through a deterministic mapping function $w = \mathbf{M}_D(D)$, which would be jointly optimized along with the generator during training. In practice, it can be implemented by assigning two different tokens for representing the (beginning of sentence) position for text generation across the two domains. This is reasonable because in auto-regressive text generation models, the process always begins with a specific token as the starting signal for generation. Consequently, the token is embedded with semantics derived from the training data during the learning process [51].

We use the personalized transformer architecture proposed in [25] for the generator G_Θ due to its effectiveness and simplicity. Given either domain, the model is optimized through the following loss:

$$\mathcal{L}_a = \ell(G_\Theta(u^D, i^D, w), \mathcal{E}^D), \quad w = \mathbf{M}_D(D), \quad (12)$$

where D denotes either the auxiliary a or target domain t , and ℓ is the cross-entropy loss function.³

Moreover, it is crucial to align the estimated user and item representations across domains. For example, learning separately on the auxiliary and target domains may not ensure that the inferred representation for user Tom's interest in *Titanic* remains consistent across domains. Hence, in the

³<https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html>

abduction step, aligning user and item representations is essential to maintaining this consistency (as we will demonstrate in Section 5). Following the cross-domain learning algorithm [9, 53, 74], we leverage textual information such as user reviews and item specifications, to eliminate the dependency on strict overlap between users and items across domains. It is important to note that this textual information differs from W : the former captures users' preferences, while the latter represents contextual text attributes. Specifically, we aggregate all reviews written by user u into a single document, denoted as $\mathcal{R}_u$, and similarly for items as $\mathcal{R}_i$. We then use the widely adopted Sentence-BERT PLM to initialize the user and item feature representations, with $\mathbf{p}_u$ for users and $\mathbf{q}_i$ for items, as follows:

$$\mathbf{p}_u = \text{PLM}(\mathcal{R}_u), \quad \mathbf{q}_i = \text{PLM}(\mathcal{R}_i). \quad (13)$$

Next, the generator G_Θ takes the representations $\mathbf{p}_u$ and $\mathbf{q}_i$ and encodes their interactions through fully visible attention mechanisms (see [25]) to generate hidden states $\mathbf{z}_u$ and $\mathbf{z}_i$. These hidden states are then passed to the decoder to generate the explanation tokens. Our goal is to promote the alignment of the encoded hidden states $\mathbf{z}_u$ and $\mathbf{z}_i$ across domains, so that users with similar semantic profiles in different domains are encoded similarly by the generator. To achieve this, we employ a user discriminator F_u and an item discriminator F_i in an adversarial learning manner [9, 53]. The discriminators need to classify whether the hidden states originate from the target or auxiliary domain, while the generator is trained to fool them. The min-max game encourages the generator to align the representations across domains during explanation generation. The adversarial loss is formulated as follows:

$$\min_{G_\Theta} \max_{F_u, F_i} \mathcal{L}_{adv} = \mathbb{E}[\log F_u(\mathbf{z}_u^a)] + \mathbb{E}[\log(1 - F_u(\mathbf{z}_u^t))] + \mathbb{E}[\log F_i(\mathbf{z}_i^a)] + \mathbb{E}[\log(1 - F_i(\mathbf{z}_i^t))], \quad (14)$$

where F_u and F_i maximize the objective to classify domain-specific representations, while G_Θ minimizes the objective by making target domain representations $\mathbf{z}_u^t, \mathbf{z}_i^t$ indistinguishable from auxiliary domain representations $\mathbf{z}_u^a, \mathbf{z}_i^a$.

Action. In this step, we replace the original text attributes $w \sim P(W|D = t)$ with alternative attributes $\tilde{w} \sim P(W|D = a)$ from the auxiliary domain. This simulates a counterfactual scenario where the same user-item interaction is described using different text attributes from auxiliary domain. The replacement is by using the domain embedding matrix $\mathbf{M}_D$, which was learned in the previous abduction step. For every interaction in the target domain, we apply this replacement as follows: $\forall(u^t, i^t, \mathcal{E}^t) \in \mathcal{X}^t, w = \mathbf{M}_D(t) \rightarrow \tilde{w} = \mathbf{M}_D(a)$, as shown in step 2 of Figure 4.

Prediction. In this step, we infer the counterfactual explanation $\tilde{\mathcal{E}}$ for each user-item interaction based on the altered text attributes $\tilde{w}$. This allows us to simulate how the explanations change under different textual patterns. Specifically, the counterfactual explanation $\tilde{\mathcal{E}}$ is generated as a sequence of tokens, where each token is predicted by maximizing the conditional probability given the user u^t , item i^t , the altered text attributes $\tilde{w}$, as follows:

$$\tilde{\mathcal{E}} = \{\tilde{e}_1, \tilde{e}_2, \dots, \tilde{e}_l\}, \quad \tilde{e}_l = \arg \max \log P(\tilde{e}_l | u^t, i^t, \tilde{w}, \tilde{e}_{<l}), \quad (15)$$

where l is the sequence length, and each token $\tilde{e}_i$ is predicted auto-regressively.

We remark that in the three-step counterfactual creation process, only the training dataset is used. This implies that, in the *Prediction* step, the explanations generated by the model for these already seen user-item pairs are free from degeneration. As discussed in Section 4.1, degeneration is due to model's reliance more on learning shortcuts than on interaction knowledge, and the problem arises when generalizing to *unseen* user-item pairs. Since the generator only processes familiar user-item pairs during counterfactuals creation, the generated counterfactual samples are considered valid and consistent. We denote the created counterfactual samples as $\mathcal{X}^c = \{u^t, i^t, \tilde{\mathcal{E}}\}$.

4.4 Model Learning and Optimization

During the optimization process, the generator is trained with two key objectives: (1) traditional MLE and (2) optimizing the causal effect of user–item interactions on explanation generation. The first objective ensures the model learns to generate coherent explanations by leveraging user interests, item features, and text attributes. The loss function for this objective is defined as follows:

$$\mathcal{L}_t = \sum_{(u,i,\mathcal{E}) \in \mathcal{X}^t} -\log P(\mathcal{E}|u, i) = \sum_{(u,i,\mathcal{E}) \in \mathcal{X}^t} -\log P(\mathcal{E}|u, i, w) |_{w=\mathbf{M}_D(t)}, \quad (16)$$

where $|_{w=\mathbf{M}_D(t)}$ denotes the w is the target domain-specific text attributes, which are inferred in previous steps and will be jointly learned with the generator.

Our second objective is to train the generator using both factual data $\mathcal{X}^t$ and counterfactual data $\mathcal{X}^c$. By incorporating counterfactual data, the generator learns to account for variations in text attributes, helping disentangle their influence. This achieves that user–item interactions become independent of text attributes, allowing the model to focus on learning the causal edges $U \rightarrow E \leftarrow I$. We denote the loss function as $\mathcal{L}_{causality}$, representing the causality-based regularization term, as follows:

$$\mathcal{L}_{causality} = \sum_{(u,i,\mathcal{E}) \in \mathcal{X}^t} -\log P(\mathcal{E}|u, i, w) |_{w=\mathbf{M}_D(t)} + \sum_{(u,i,\tilde{\mathcal{E}}) \in \mathcal{X}^c} -\log P(\tilde{\mathcal{E}}|u, i, \tilde{w}) |_{\tilde{w}=\mathbf{M}_D(a)}. \quad (17)$$

We combine both objectives into the final loss function $\mathcal{L}_{total}$ as shown below. For clarity, we omit the conditions $|_{w=\mathbf{M}_D(t)}$ and $|_{\tilde{w}=\mathbf{M}_D(a)}$ in the expressions.

$$\begin{aligned} \mathcal{L}_{total} &= \mathcal{L}_t + \eta \cdot \mathcal{L}_{causality} \\ &= \sum_{(u,i,\mathcal{E}) \in \mathcal{X}^t} -\log P(\mathcal{E}|u, i, w) + \eta \cdot \left(\sum_{(u,i,\mathcal{E}) \in \mathcal{X}^t} -\log P(\mathcal{E}|u, i, w) + \sum_{(u,i,\tilde{\mathcal{E}}) \in \mathcal{X}^c} -\log P(\tilde{\mathcal{E}}|u, i, \tilde{w}) \right) \\ &= \sum_{(u,i,\mathcal{E}) \in \mathcal{X}^t} -(1 + \eta) \log P(\mathcal{E}|u, i, w) + \sum_{(u,i,\tilde{\mathcal{E}}) \in \mathcal{X}^c} -\eta \log P(\tilde{\mathcal{E}}|u, i, \tilde{w}) \\ &\approx \sum_{(u,i,\mathcal{E}) \in \mathcal{X}^t} -\log P(\mathcal{E}|u, i, w) + \sum_{(u,i,\tilde{\mathcal{E}}) \in \mathcal{X}^c} -\eta \log P(\tilde{\mathcal{E}}|u, i, \tilde{w}), \end{aligned} \quad (18)$$

where η is a hyper-parameter controlling the weight of the causality loss. This equation can be interpreted from two perspectives. First, the weighted summation of the loss function means that we incorporate causality as a regularization term. This regularization guides the model to generate coherent texts while learning disentangled causal effects of user–item interactions on explanations. Second, from the data augmentation perspective, the equation shows that we assign a learning weight to train the model simultaneously on an augmented dataset of counterfactual samples, thereby enhancing its generalizability. In summary, we name our approach as **domain for counterfactual reasoning (D4C)**, and the complete learning procedure is outlined in Algorithm 1.

5 Experiments

5.1 Experimental Setup

5.1.1 Datasets. We conduct experiments on five diverse datasets to evaluate the performance of our proposed method against existing methods. Three datasets are from Amazon [17], specifically the *CDs and Vinyl*, *Electronics*, and *Movies and TV* categories. Additionally, we include two datasets from other platforms: TripAdvisor and Yelp [24]. The *TripAdvisor* dataset consists of travel and hotel reviews, while the *Yelp* dataset, part of the Yelp Challenge, provides user ratings and reviews for various businesses. These datasets are widely used in previous works [7, 13, 25, 74].

Algorithm 1: Learning Algorithm of D4C

```

1 Input: Target domain training set  $\mathcal{X}^t$ , auxiliary domain training set  $\mathcal{X}^a$ 
2 Output: Trained generator  $G_\Theta$ 
3 Initialize the parameters of the generator  $G_\Theta$ , discriminators  $F_u, F_i$ , and domain embeddings  $M_D$ 
4 Initialize counterfactual training set  $\mathcal{X}^c = \emptyset$ 
5 while not converged do
6   for each  $(u^a, i^a, \mathcal{E}^a) \in \mathcal{X}^a$  and  $(u^t, i^t, \mathcal{E}^t) \in \mathcal{X}^t$  do
7     Update generator  $G_\Theta$  to minimize the abduction loss  $\mathcal{L}_a$  in Equation (12).
8     Update discriminators  $F_u, F_i$  to maximize the adversarial loss  $\mathcal{L}_{adv}$  in Equation (14).
9     Update generator  $G_\Theta$  to minimize the adversarial loss  $\mathcal{L}_{adv}$  in Equation (14).
10 for each  $(u^t, i^t, \mathcal{E}^t) \in \mathcal{X}^t$  do
11   Replace text attribute  $w$  with  $\tilde{w} = M_D(a)$ 
12   Generate counterfactual explanation  $\tilde{\mathcal{E}} = G_\Theta(u^t, i^t, \tilde{w})$ 
13   Add counterfactual sample  $(u^t, i^t, \tilde{\mathcal{E}})$  to  $\mathcal{X}^c$ 
14 Retrain the generator  $G_\Theta$  using Equation (18).

```

Table 2. Statistics of the Datasets

Dataset	Num. of Users	Num. of Items	Num. of Interactions	Avg. Length of Explanation	Density (%)
AM-CDs and Vinyl	12,841	15,604	196,002	13.93	0.098
AM-Electronics	28,032	15,796	245,519	13.60	0.055
AM-Movies and TV	21,014	18,962	355,415	14.18	0.089
TripAdvisor	9,765	6,280	288,020	12.99	0.469
Yelp	27,128	20,256	646,518	12.23	0.118

For data pre-processing, we follow the procedures in [25, 74], which include the following steps: (1) *Explanation Extraction*: Explanations are extracted from review texts for each interaction using a commonly used explanation extraction tool.⁴ When multiple explanations are available in a review, the longest one is selected as the ground-truth, and reviews without explanations are excluded. (2) *Data Cleaning*: Users and items with fewer than five interactions are removed to ensure data quality. (3) *Explanation Length*: Explanations are cropped to a maximum sequence length of 25 words for computational efficiency. (4) *Data Splitting*: The data are divided into training, validation, and test sets in an 8:1:1 ratio. Table 2 presents the statistics of the datasets.

5.1.2 Evaluation Metrics. To evaluate the quality of explanation generation, we use metrics from multiple perspectives: (a) *N-Gram Matching*: This metric assesses the similarity between the generated text and reference n-grams. We utilize ROUGE [29] and BLEU [39], where BLEU primarily measures precision, and ROUGE emphasizes recall. Additionally, METEOR [2] is used, combining precision and recall as a harmonic mean, with greater weight placed on recall. (b) *Diversity*: The diversity of generated explanations is evaluated using the Distinct score [23]. Distinct-1 and Distinct-2 quantify diversity by calculating the ratio of unique unigrams and bigrams, respectively, in the generated sentences. While our focus is explanation generation, we also measure the accuracy of rating prediction to ensure a thorough evaluation. We use **mean absolute error (MAE)** and RMSE to evaluate the accuracy of recommendations. For explanation metrics, higher scores indicate

⁴<https://github.com/lileipisces/Sentires-Guide>

better performance, while for recommendation metrics, lower RMSE and MAE scores indicate better performance.

5.1.3 Compared Methods. We introduce the following baselines used in our experiments:

- *DualPC* [45] employs a dual-form approach to align ratings with explanations more closely.
- *PETER* [25] is a personalized transformer model that generates explanations with linking ID- and word-embeddings through a novel context prediction task.
- *ReXPlug* [16] utilizes controlled text generation based on the GPT-2 model for generating better explanations.
- *ERRA* [7] enhances the PETER model through retrieving relevant reviews and enhanced aspect learning.

Since our method utilizes an auxiliary domain, for a fair comparison, we adapt methods for cross-domain learning that also employ auxiliary domains, as follows:

- *PEFT*: namely PETER Fine Tuning. We pre-train the PETER in the auxiliary domain and then transfer the learned parameters of personalized transformers to the target domain for fine-tuning. By comparing PEFT with the original PETER, we can evaluate the difficulties in knowledge transfer in explanation generation task.
- *PEPLER* [26] treats user and item IDs as prompts and uses a GPT-2 model with prompt tuning to generate explanations. PEPLER has two variants: discrete and continuous prompt learning. We select continuous prompt learning due to its superior performance. For cross-domain learning, we first optimize the prompt parameters by jointly training on the auxiliary and target domains, and then tune on the target domain.
- *AdaReX* [74] leverages aspect sequences to learn shared knowledge between auxiliary and target domains. We adapt it by optimizing the auxiliary and target domain generators simultaneously for each domain.

5.1.4 Implementation Details. All models are implemented using PyTorch⁵ and run on NVIDIA RTX 4090 GPUs. The following implementation details apply to all models unless otherwise specified: (1) *Vocabulary Construction*: To build a common vocabulary for both auxiliary and target domains, we use the SentencePiece algorithm [21], which segments words into sub-words without requiring a fixed vocabulary size. We apply SentencePiece for tokenizer construction across all models, except for ReXPlug and PEPLER, as these models rely on the pre-trained GPT-2 tokenizer. (2) *Training Procedure*: Training is conducted for a maximum of 50 epochs. The learning rate is dynamically adjusted based on validation loss; if the validation loss increases, the learning rate is halved. Early stopping with a patience of 5 epochs is applied to prevent overfitting. The batch size is set to 128, and the Adam optimizer [20] is used for optimization. (3) *Hyper-Parameter Tuning*: Grid search is employed to tune the following hyper-parameters: learning rates in $\{5e-5, 1e-4, 5e-4, 1e-3, 5e-3, 1e-2\}$; embedding sizes in $\{100, 200, 300, 400, 500\}$; and the number of layers in $\{2, 4, 6, 8\}$. For specific models, λ_1 and λ_2 are set to 0.2 in DualPC; the aspect sequence length is fixed at 25 in AdaReX; and for D4C, the hyper-parameter η is tuned in $\{0.0001, 0.001, 0.01, 0.1\}$. See Section 5.4.1 for a detailed analysis of its impact. (4) *Model Selection*: BLEU-4 scores are used to select optimal hyper-parameters by evaluating the model on the validation set, as they better capture the quality and robustness of the generated text. (5) *Results Reporting*: To ensure reliable results, we report the average performance of the final model across five independent runs, each with a different random seed. (6) *Explanation Generation*: A greedy algorithm is employed for generating explanations across all models.

⁵<https://pytorch.org/>

Table 3. Overall Performance on the *AM-CDs* Dataset

Methods	ROUGE-1	ROUGE-L	BLEU-1	BLEU-2	BLEU-3	BLEU-4	METEOR	DIST-1	DIST-2	RMSE	MAE
Single Domain											
DualPC	12.50 $\pm$ 0.45	9.67 $\pm$ 0.31	9.32 $\pm$ 0.34	3.69 $\pm$ 0.18	1.60 $\pm$ 0.10	0.88 $\pm$ 0.08	7.98 $\pm$ 0.37	0.01 $\pm$ 0	0.02 $\pm$ 0	0.9563 $\pm$ 0.0108	0.6736 $\pm$ 0.0084
PETER	11.62 $\pm$ 0.36	9.07 $\pm$ 0.25	10.42 $\pm$ 0.88	3.86 $\pm$ 0.25	1.77 $\pm$ 0.11	0.99 $\pm$ 0.08	8.41 $\pm$ 0.28	0.78 $\pm$ 0.49	2.64 $\pm$ 1.43	0.9191 $\pm$ 0.0055	0.6577 $\pm$ 0.0110
ReXPlug	13.16 $\pm$ 0.70	10.18 $\pm$ 0.63	9.85 $\pm$ 1.90	3.86 $\pm$ 0.25	1.64 $\pm$ 0.20	0.88 $\pm$ 0.08	8.13 $\pm$ 0.49	0.59 $\pm$ 0.23	2.53 $\pm$ 1.09	0.9580 $\pm$ 0.0030	0.6751 $\pm$ 0.0029
ERRA	12.72 $\pm$ 0.72	9.92 $\pm$ 0.53	11.53 $\pm$ 1.22	4.32 $\pm$ 0.39	1.99 $\pm$ 0.15	1.12 $\pm$ 0.08	8.88 $\pm$ 0.44	0.42 $\pm$ 0.11	1.46 $\pm$ 0.39	0.9128 $\pm$ 0.0086	0.6592 $\pm$ 0.0037
Cross-Domain											
PEFT (E)	12.37 $\pm$ 0.61	9.57 $\pm$ 0.36	11.68 $\pm$ 0.81	4.34 $\pm$ 0.21	2.00 $\pm$ 0.08	1.13 $\pm$ 0.05	9.17 $\pm$ 0.43	0.83 $\pm$ 0.27	2.88 $\pm$ 1.05	0.9091 $\pm$ 0.0023	0.6653 $\pm$ 0.0102
PEFT (M)	12.41 $\pm$ 0.58	9.69 $\pm$ 0.37	11.67 $\pm$ 1.43	4.29 $\pm$ 0.52	1.95 $\pm$ 0.22	1.10 $\pm$ 0.11	8.98 $\pm$ 0.61	0.53 $\pm$ 0.17	1.75 $\pm$ 0.60	0.9050 $\pm$ 0.0058	0.6767 $\pm$ 0.0196
PEPLER (E)	12.59 $\pm$ 0.18	9.78 $\pm$ 0.15	11.57 $\pm$ 0.21	4.08 $\pm$ 0.07	1.79 $\pm$ 0.06	0.98 $\pm$ 0.05	8.87 $\pm$ 0.17	0.80 $\pm$ 0.15	3.21 $\pm$ 0.61	0.9008 $\pm$ 0.0001	0.6561 $\pm$ 0.0001
PEPLER (M)	12.80 $\pm$ 0.28	9.71 $\pm$ 0.24	11.67 $\pm$ 0.50	4.07 $\pm$ 0.11	1.78 $\pm$ 0.04	0.97 $\pm$ 0.03	8.94 $\pm$ 0.34	0.70 $\pm$ 0.26	2.94 $\pm$ 1.24	0.9003 $\pm$ 0.0070	0.6568 $\pm$ 0.0051
AdaReX (E)	13.42 $\pm$ 0.27	10.37 $\pm$ 0.18	12.64 $\pm$ 0.49	4.54 $\pm$ 0.16	2.00 $\pm$ 0.09	1.08 $\pm$ 0.07	9.48 $\pm$ 0.41	0.67 $\pm$ 0.16	2.62 $\pm$ 0.28	0.9188 $\pm$ 0.0049	0.6505 $\pm$ 0.0122
AdaReX (M)	13.38 $\pm$ 0.72	10.40 $\pm$ 0.53	12.60 $\pm$ 1.15	4.67 $\pm$ 0.43	2.09 $\pm$ 0.19	1.15 $\pm$ 0.12	9.52 $\pm$ 0.53	0.89 $\pm$ 0.11	3.13 $\pm$ 0.37	0.9178 $\pm$ 0.0051	0.6582 $\pm$ 0.0124
D4C (E)	13.50 $\pm$ 0.46	10.61 $\pm$ 0.19	12.71 $\pm$ 0.19	4.69 $\pm$ 0.29	2.23 $\pm$ 0.14	1.21 $\pm$ 0.12	9.69 $\pm$ 0.35	0.93 $\pm$ 0.27	3.57 $\pm$ 1.13	0.9130 $\pm$ 0.0125	0.6594 $\pm$ 0.0131
D4C (M)	13.41 $\pm$ 0.18	10.49 $\pm$ 0.33	12.84 $\pm$ 0.50	4.85 $\pm$ 0.26	2.29 $\pm$ 0.18	1.26 $\pm$ 0.10	9.82 $\pm$ 0.37	0.98 $\pm$ 0.30	4.07 $\pm$ 1.09	0.9179 $\pm$ 0.0118	0.6601 $\pm$ 0.0144

The first group of methods is focused on a single domain, while the second group employs cross-domain learning using auxiliary domains. Inside the parentheses, “E” and “M” refer to *AM-Electronics* and *AM-Movies* as the auxiliary domains, respectively. Bold values indicate the best performance.

Table 4. Overall Performance on the *AM-Electronics* Dataset

Methods	ROUGE-1	ROUGE-L	BLEU-1	BLEU-2	BLEU-3	BLEU-4	METEOR	DIST-1	DIST-2	RMSE	MAE
Single Domain											
DualPC	10.43 $\pm$ 1.30	8.13 $\pm$ 1.07	9.79 $\pm$ 1.87	2.62 $\pm$ 0.65	0.81 $\pm$ 0.22	0.35 $\pm$ 0.04	7.61 $\pm$ 0.68	0.01 $\pm$ 0.01	0.02 $\pm$ 0.01	1.1018 $\pm$ 0.0484	0.7876 $\pm$ 0.0212
PETER	12.68 $\pm$ 1.27	9.55 $\pm$ 0.85	12.04 $\pm$ 1.04	3.87 $\pm$ 0.47	1.44 $\pm$ 0.23	0.65 $\pm$ 0.12	8.89 $\pm$ 0.50	0.64 $\pm$ 0.23	2.00 $\pm$ 0.71	1.0550 $\pm$ 0.0074	0.7618 $\pm$ 0.0049
ReXPlug	10.83 $\pm$ 2.03	8.69 $\pm$ 1.74	9.36 $\pm$ 3.50	2.76 $\pm$ 1.15	0.85 $\pm$ 0.32	0.30 $\pm$ 0.14	7.93 $\pm$ 2.27	0.53 $\pm$ 0.26	1.11 $\pm$ 0.17	1.0868 $\pm$ 0.0093	0.7809 $\pm$ 0.0166
ERRA	13.27 $\pm$ 0.73	10.01 $\pm$ 0.56	13.25 $\pm$ 0.89	4.30 $\pm$ 0.35	1.56 $\pm$ 0.14	0.69 $\pm$ 0.06	9.55 $\pm$ 0.76	0.64 $\pm$ 0.14	2.04 $\pm$ 0.39	1.0506 $\pm$ 0.0052	0.7577 $\pm$ 0.0117
Cross-Domain											
PEFT (C)	12.67 $\pm$ 0.19	9.41 $\pm$ 0.19	12.04 $\pm$ 0.50	3.85 $\pm$ 0.15	1.43 $\pm$ 0.07	0.62 $\pm$ 0.06	8.87 $\pm$ 0.29	0.69 $\pm$ 0.30	1.29 $\pm$ 0.19	1.0459 $\pm$ 0.0098	0.7727 $\pm$ 0.0115
PEFT (M)	12.89 $\pm$ 0.39	10.01 $\pm$ 0.33	13.85 $\pm$ 0.50	4.20 $\pm$ 0.21	1.32 $\pm$ 0.09	0.54 $\pm$ 0.04	9.53 $\pm$ 0.27	0.33 $\pm$ 0.07	1.07 $\pm$ 0.30	1.0508 $\pm$ 0.0046	0.7679 $\pm$ 0.0063
PEPLER (C)	14.01 $\pm$ 0.25	10.09 $\pm$ 0.16	12.48 $\pm$ 0.21	3.87 $\pm$ 0.11	1.36 $\pm$ 0.05	0.54 $\pm$ 0.02	10.29 $\pm$ 0.22	0.24 $\pm$ 0.01	1.21 $\pm$ 0.05	1.0348 $\pm$ 0.0001	0.7657 $\pm$ 0.0005
PEPLER (M)	13.91 $\pm$ 0.23	10.47 $\pm$ 0.13	12.45 $\pm$ 0.13	3.85 $\pm$ 0.06	1.37 $\pm$ 0.03	0.55 $\pm$ 0.01	10.26 $\pm$ 0.16	0.25 $\pm$ 0.01	1.22 $\pm$ 0.05	1.0349 $\pm$ 0.0002	0.7661 $\pm$ 0.0004
AdaReX (C)	13.27 $\pm$ 0.81	9.98 $\pm$ 0.52	13.11 $\pm$ 1.06	4.25 $\pm$ 0.42	1.53 $\pm$ 0.16	0.66 $\pm$ 0.06	9.60 $\pm$ 0.41	0.50 $\pm$ 0.11	1.63 $\pm$ 0.42	1.0662 $\pm$ 0.0104	0.7771 $\pm$ 0.0158
AdaReX (M)	13.53 $\pm$ 0.93	9.70 $\pm$ 0.73	13.44 $\pm$ 1.07	3.98 $\pm$ 0.24	1.43 $\pm$ 0.14	0.57 $\pm$ 0.09	9.42 $\pm$ 0.39	0.55 $\pm$ 0.10	1.87 $\pm$ 0.45	1.0622 $\pm$ 0.0194	0.7726 $\pm$ 0.0115
D4C (C)	14.02 $\pm$ 0.79	10.42 $\pm$ 0.45	14.16 $\pm$ 0.53	4.39 $\pm$ 0.24	1.69 $\pm$ 0.11	0.78 $\pm$ 0.09	10.32 $\pm$ 0.38	0.68 $\pm$ 0.17	2.21 $\pm$ 0.48	1.0847 $\pm$ 0.0140	0.7836 $\pm$ 0.0103
D4C (M)	13.56 $\pm$ 0.83	10.19 $\pm$ 0.47	13.58 $\pm$ 0.61	4.33 $\pm$ 0.30	1.68 $\pm$ 0.09	0.79 $\pm$ 0.08	10.24 $\pm$ 0.34	0.85 $\pm$ 0.41	2.39 $\pm$ 0.71	1.0794 $\pm$ 0.0089	0.7756 $\pm$ 0.0049

“C” and “M” refer to *AM-CDs*, and *AM-Movies* as the auxiliary domains, respectively. Bold values indicate the best performance.

5.2 Overall Performance

The comprehensive performance are detailed in Tables 3 to 7. From the tables, we summarize the observations as follows:

- Cross-domain models generally achieve better performance. In most datasets, models utilizing cross-domain information, such as PEPLER, AdaReX, and D4C, consistently outperform single-domain baselines like DualPC, PETER, and ReXPlug in explanation generation. This is expected, as cross-domain models leverage auxiliary knowledge, improving generalization and providing a deeper understanding of item–user interactions.
- PEPLER and AdaReX demonstrate strong cross-domain capabilities. Among the compared methods, PEFT (pre-train fine-tuning) performs the weakest in explanation generation. In contrast, PEPLER (prompt-tuning) and AdaReX (domain adaptation) achieve higher scores in text metrics by more effectively capturing and transferring domain-specific knowledge.
- D4C consistently achieves the best performance across datasets, outperforming other models in explanation metrics such as ROUGE, BLEU, and DIST. This demonstrates the effectiveness of counterfactual data augmentation in enhancing overall performance.

Table 5. Overall Performance on the *AM-Movies* Dataset

Methods	ROUGE-1	ROUGE-L	BLEU-1	BLEU-2	BLEU-3	BLEU-4	METEOR	DIST-1	DIST-2	RMSE	MAE
Single Domain											
DualPC	11.83 $\pm$ 0.35	9.10 $\pm$ 0.22	9.21 $\pm$ 0.27	3.04 $\pm$ 0.14	1.23 $\pm$ 0.08	0.63 $\pm$ 0.05	7.94 $\pm$ 0.31	0.03 $\pm$ 0	0.06 $\pm$ 0.05	1.0504 $\pm$ 0.013	0.7567 $\pm$ 0.0078
PETER	14.35 $\pm$ 0.59	10.92 $\pm$ 0.35	13.24 $\pm$ 0.93	4.82 $\pm$ 0.30	2.18 $\pm$ 0.10	1.21 $\pm$ 0.05	9.75 $\pm$ 0.38	0.48 $\pm$ 0.16	1.85 $\pm$ 0.78	0.9959 $\pm$ 0.0072	0.7411 $\pm$ 0.0101
ReXPlug	10.86 $\pm$ 2.79	8.30 $\pm$ 2.27	6.83 $\pm$ 3.37	2.10 $\pm$ 1.18	0.72 $\pm$ 0.36	0.30 $\pm$ 0.13	6.55 $\pm$ 1.37	0.39 $\pm$ 0.11	1.37 $\pm$ 0.40	1.0236 $\pm$ 0.0104	0.7479 $\pm$ 0.0050
ERRA	13.71 $\pm$ 1.04	10.47 $\pm$ 0.69	12.14 $\pm$ 1.97	4.41 $\pm$ 0.56	1.99 $\pm$ 0.14	1.14 $\pm$ 0.05	9.42 $\pm$ 0.55	0.33 $\pm$ 0.04	1.26 $\pm$ 0.13	1.0025 $\pm$ 0.0093	0.7413 $\pm$ 0.0076
Cross-Domain											
PEFT (C)	13.79 $\pm$ 0.77	9.93 $\pm$ 0.13	12.24 $\pm$ 0.34	4.72 $\pm$ 0.48	1.90 $\pm$ 0.05	1.05 $\pm$ 0.04	9.56 $\pm$ 0.54	0.57 $\pm$ 0.20	2.23 $\pm$ 0.46	0.9944 $\pm$ 0.0023	0.7339 $\pm$ 0.0090
PEFT (E)	13.89 $\pm$ 0.49	9.97 $\pm$ 0.07	12.22 $\pm$ 0.23	4.90 $\pm$ 0.22	1.88 $\pm$ 0.07	1.04 $\pm$ 0.05	10.14 $\pm$ 0.42	0.45 $\pm$ 0.12	1.59 $\pm$ 0.46	0.9921 $\pm$ 0.0031	0.7471 $\pm$ 0.0136
PEPLER (C)	13.66 $\pm$ 0.09	11.20 $\pm$ 0.33	12.80 $\pm$ 0.54	5.25 $\pm$ 0.09	2.20 $\pm$ 0.18	1.29 $\pm$ 0.09	10.21 $\pm$ 0.03	0.59 $\pm$ 0.03	2.98 $\pm$ 0.13	0.9919 $\pm$ 0.0001	0.7598 $\pm$ 0.0002
PEPLER (E)	13.62 $\pm$ 0.19	10.52 $\pm$ 1.46	12.65 $\pm$ 1.46	4.94 $\pm$ 0.16	2.23 $\pm$ 0.10	1.17 $\pm$ 0.06	10.13 $\pm$ 0.17	0.61 $\pm$ 0.05	3.01 $\pm$ 0.22	0.9920 $\pm$ 0.0001	0.7598 $\pm$ 0.0003
AdaReX (C)	13.97 $\pm$ 0.48	10.60 $\pm$ 0.25	12.53 $\pm$ 0.65	4.62 $\pm$ 0.24	2.11 $\pm$ 0.15	1.20 $\pm$ 0.12	9.94 $\pm$ 0.36	0.44 $\pm$ 0.10	2.29 $\pm$ 0.32	0.9987 $\pm$ 0.0056	0.7305 $\pm$ 0.0041
AdaReX (E)	14.21 $\pm$ 0.56	10.89 $\pm$ 0.39	13.31 $\pm$ 0.47	4.81 $\pm$ 0.17	2.11 $\pm$ 0.08	1.23 $\pm$ 0.05	9.97 $\pm$ 0.29	0.52 $\pm$ 0.18	2.24 $\pm$ 0.60	0.9988 $\pm$ 0.0054	0.7360 $\pm$ 0.0099
D4C (C)	14.45 $\pm$ 0.43	11.08 $\pm$ 0.24	13.41 $\pm$ 0.36	4.95 $\pm$ 0.16	2.26 $\pm$ 0.11	1.31 $\pm$ 0.07	10.19 $\pm$ 0.14	1.09 $\pm$ 0.13	4.30 $\pm$ 0.46	1.0045 $\pm$ 0.0108	0.7352 $\pm$ 0.0089
D4C (E)	14.54 $\pm$ 0.26	11.25 $\pm$ 0.20	13.49 $\pm$ 0.47	5.12 $\pm$ 0.18	2.33 $\pm$ 0.13	1.30 $\pm$ 0.05	10.13 $\pm$ 0.17	0.98 $\pm$ 0.12	3.97 $\pm$ 0.51	1.0071 $\pm$ 0.0122	0.7376 $\pm$ 0.0098

“C” and “E” refer to *AM-CDs*, and *AM-Electronics* as the auxiliary domains, respectively. Bold values indicate the best performance.

Table 6. Overall Performance on the *TripAdvisor* Dataset

Methods	ROUGE-1	ROUGE-L	BLEU-1	BLEU-2	BLEU-3	BLEU-4	METEOR	DIST-1	DIST-2	RMSE	MAE
DualPC	16.41 $\pm$ 0.69	12.59 $\pm$ 0.17	14.80 $\pm$ 0.30	5.62 $\pm$ 0.16	2.10 $\pm$ 0.05	0.85 $\pm$ 0.03	10.35 $\pm$ 0.72	0.04 $\pm$ 0.01	0.09 $\pm$ 0.03	0.8290 $\pm$ 0.0111	0.6260 $\pm$ 0.0065
PETER	16.72 $\pm$ 0.26	12.79 $\pm$ 0.15	15.64 $\pm$ 0.84	5.85 $\pm$ 0.31	2.43 $\pm$ 0.17	1.09 $\pm$ 0.09	10.46 $\pm$ 0.42	0.58 $\pm$ 0.11	1.89 $\pm$ 0.27	0.8193 $\pm$ 0.0031	0.6205 $\pm$ 0.0069
ReXPlug	17.40 $\pm$ 1.49	12.78 $\pm$ 1.22	12.98 $\pm$ 4.16	5.01 $\pm$ 1.60	1.94 $\pm$ 0.54	0.74 $\pm$ 0.21	10.29 $\pm$ 2.04	0.25 $\pm$ 0.05	1.40 $\pm$ 0.30	0.8571 $\pm$ 0.0178	0.6541 $\pm$ 0.0118
ERRA	16.76 $\pm$ 0.17	13.14 $\pm$ 0.13	16.07 $\pm$ 0.38	5.92 $\pm$ 0.19	2.47 $\pm$ 0.09	1.10 $\pm$ 0.06	10.10 $\pm$ 0.15	0.61 $\pm$ 0.22	2.24 $\pm$ 0.62	0.7896 $\pm$ 0.0005	0.6091 $\pm$ 0.0025
PEFT	15.93 $\pm$ 0.07	12.16 $\pm$ 0.03	14.21 $\pm$ 0.13	5.34 $\pm$ 0.07	2.25 $\pm$ 0.03	1.00 $\pm$ 0.02	9.20 $\pm$ 0.02	0.22 $\pm$ 0.04	0.81 $\pm$ 0.20	0.8000 $\pm$ 0.0021	0.6180 $\pm$ 0.0023
PEPLER	16.07 $\pm$ 0.13	11.72 $\pm$ 0.14	13.26 $\pm$ 0.25	4.82 $\pm$ 0.11	2.00 $\pm$ 0.04	0.87 $\pm$ 0.02	9.59 $\pm$ 0.11	0.28 $\pm$ 0.05	1.54 $\pm$ 0.33	0.7910 $\pm$ 0.0001	0.6084 $\pm$ 0.0015
AdaReX	16.77 $\pm$ 0.33	13.18 $\pm$ 0.17	16.27 $\pm$ 0.27	6.20 $\pm$ 0.16	2.48 $\pm$ 0.06	1.13 $\pm$ 0.04	10.31 $\pm$ 0.27	0.60 $\pm$ 0.13	1.82 $\pm$ 0.46	0.7904 $\pm$ 0.0006	0.6058 $\pm$ 0.0037
D4C	17.08 $\pm$ 0.42	13.18 $\pm$ 0.29	16.70 $\pm$ 0.35	6.22 $\pm$ 0.21	2.57 $\pm$ 0.12	1.15 $\pm$ 0.06	10.76 $\pm$ 0.33	0.71 $\pm$ 0.16	2.59 $\pm$ 0.38	0.8014 $\pm$ 0.0109	0.6132 $\pm$ 0.0023

Bold values indicate the best performance.

Table 7. Overall Performance on the *Yelp* Dataset

Methods	ROUGE-1	ROUGE-L	BLEU-1	BLEU-2	BLEU-3	BLEU-4	METEOR	DIST-1	DIST-2	RMSE	MAE
DualPC	10.48 $\pm$ 0.51	8.11 $\pm$ 0.42	10.33 $\pm$ 0.69	2.71 $\pm$ 0.25	0.95 $\pm$ 0.09	0.41 $\pm$ 0.06	6.31 $\pm$ 0.54	0.05 $\pm$ 0.02	0.04 $\pm$ 0.01	1.0699 $\pm$ 0.0109	0.8264 $\pm$ 0.0071
PETER	12.34 $\pm$ 0.61	9.55 $\pm$ 0.46	12.30 $\pm$ 0.49	3.76 $\pm$ 0.21	1.47 $\pm$ 0.09	0.62 $\pm$ 0.04	7.87 $\pm$ 0.34	0.79 $\pm$ 0.31	2.65 $\pm$ 0.73	1.0459 $\pm$ 0.0016	0.8095 $\pm$ 0.0031
ReXPlug	9.45 $\pm$ 1.96	7.51 $\pm$ 1.80	9.45 $\pm$ 1.78	2.32 $\pm$ 0.63	0.72 $\pm$ 0.19	0.30 $\pm$ 0.08	6.22 $\pm$ 1.72	0.37 $\pm$ 0.08	1.73 $\pm$ 0.48	1.0414 $\pm$ 0.0049	0.8141 $\pm$ 0.0037
ERRA	11.56 $\pm$ 0.47	9.02 $\pm$ 0.35	10.66 $\pm$ 0.48	3.24 $\pm$ 0.17	1.29 $\pm$ 0.07	0.58 $\pm$ 0.03	7.00 $\pm$ 0.31	0.47 $\pm$ 0.11	2.49 $\pm$ 0.81	1.0519 $\pm$ 0.0125	0.8106 $\pm$ 0.0101
PEFT	11.32 $\pm$ 0.39	8.79 $\pm$ 0.34	11.54 $\pm$ 0.44	3.39 $\pm$ 0.20	1.30 $\pm$ 0.09	0.56 $\pm$ 0.06	7.37 $\pm$ 0.27	1.20 $\pm$ 0.23	3.90 $\pm$ 0.75	1.0393 $\pm$ 0.0041	0.8085 $\pm$ 0.0035
PEPLER	12.13 $\pm$ 0.17	9.19 $\pm$ 0.28	10.67 $\pm$ 0.52	3.38 $\pm$ 0.15	1.39 $\pm$ 0.05	0.60 $\pm$ 0.03	8.07 $\pm$ 0.24	0.51 $\pm$ 0.15	3.84 $\pm$ 0.98	1.0358 $\pm$ 0.0004	0.7984 $\pm$ 0.0001
AdaReX	12.36 $\pm$ 0.41	9.61 $\pm$ 0.32	12.31 $\pm$ 0.50	3.78 $\pm$ 0.21	1.49 $\pm$ 0.10	0.64 $\pm$ 0.05	7.98 $\pm$ 0.21	0.60 $\pm$ 0.36	3.05 $\pm$ 1.02	1.0561 $\pm$ 0.0047	0.8050 $\pm$ 0.0035
D4C	13.22 $\pm$ 0.55	10.29 $\pm$ 0.41	12.75 $\pm$ 0.32	4.03 $\pm$ 0.18	1.62 $\pm$ 0.07	0.72 $\pm$ 0.05	8.23 $\pm$ 0.34	1.22 $\pm$ 0.29	3.95 $\pm$ 0.73	1.0472 $\pm$ 0.0098	0.8075 $\pm$ 0.0064

Bold values indicate the best performance.

—Tradeoff between accuracy and explanation diversity: We observe a tradeoff between accuracy and diversity in explanations. For example, some models achieve better BLEU scores but do not necessarily show good performance in DIST scores. Nevertheless, D4C balances this tradeoff effectively, generating coherent explanations with high BLEU scores while also achieving strong DIST scores to ensure diversity. This highlights the strength of our approach in addressing the degeneration issue.

Then, we analyze each dataset in detail.

AM-CDs. Table 3 shows the results of the compared methods. It is observed that D4C achieves the best performance in ROUGE-1, ROUGE-L, METEOR, and all BLEU scores (BLEU-1 to BLEU-4), showcasing its ability to generate more coherent explanations. For example, D4C outperforms the strongest baseline, AdaReX, with a 9.6% improvement in both BLEU-3 and BLEU-4. Furthermore,

D4C demonstrates notable gains in DIST-1 and DIST-2, surpassing the best baseline by 10.1% and 26.8%, respectively. These results highlight that D4C effectively addresses the issue of repetitive explanation generation, producing more diverse and personalized explanations.

AM-Electronics. Table 4 presents the results on *AM-Electronics* dataset. From the table, our D4C model shows marked improvements in BLEU-3, BLEU-4, and DIST scores. In particular, BLEU-3 increased by 8.3%, BLEU-4 by 14.5%, while Distinct-1 and Distinct-2 rose by 23.2% and 17.2%, respectively. A notable trend across all cross-domain models is observed: using *AM-CDs* as the auxiliary domain tends to result in higher text quality metrics (e.g., ROUGE, BLEU, METEOR), whereas using *AM-Movies* leads to better explanation diversity (e.g., DIST-1 and DIST-2). We attribute this to the degree of similarity between domains. To investigate further, we aggregated text from each domain and calculated the cosine similarity between them. The results reveal that the cosine similarity between *AM-CDs* and *AM-Electronics* is 0.07, while between *AM-Movies* and *AM-Electronics* it is 0.06. This finding suggests that more similar domains are better for improving text quality, while more diverse domains are beneficial for enhancing explanation diversity.

AM-Movies. Table 5 shows the results on *AM-Movies* dataset. We observe that D4C brings modest improvements in text quality, with a 4.5% increase in BLEU-3 and a 1.6% increase in BLEU-4. However, it achieves significant gains in DIST-1 and DIST-2, with increases of 78.7% and 42.9%, respectively. The marginal improvement in text quality scores is likely due to the original movie dataset containing very similar explanations, with most sentences describing movies being alike. As a result, many models achieve high scores by simply generating similar explanations. However, overly similar explanations are often less useful in practice. The significant improvements on diversity metrics highlights the strength of our model, as D4C generates more diverse explanations while maintaining high text quality, demonstrating its advantage in personalized explanation generation.

TripAdvisor. Table 6 presents the results on the *TripAdvisor* dataset. Our approach surpasses others in BLEU-1, BLEU-2, BLEU-3, BLEU-4, METEOR, Distinct-1, and Distinct-2 by 2.6%, 0.3%, 3.6%, 1.8%, 2.9%, 16.4%, and 15.6%, respectively. Notably, single-domain methods like PETER and ERRA perform well in both explanation quality and diversity, which can be attributed to the high density of the *TripAdvisor* dataset. As shown in Table 2, the dataset has a density of 0.469%, with an average of 29 training samples per user and 46 per item. However, obtaining such rich explanation data is unrealistic in practical settings, as most users are silent, and most items belong to the long tail. This highlights a limitation of existing methods, which largely rely on rich explanation datasets and overlook the practical scenarios with sparse explanations. In such cases, our method effectively utilizes auxiliary domains to guide the learning of user and item features in the target domain, enhancing the model's ability to learn meaningful representations and improving overall explanation performance.

Yelp. The results of the methods on *Yelp* are shown in Table 7. Our method achieves the best performance across most metrics, with improvements of 7.0% in ROUGE-1, 7.1% in ROUGE-L, 3.6% in BLEU-1, 6.6% in BLEU-2, 8.7% in BLEU-3, 12.5% in BLEU-4, 3.1% in METEOR, 1.7% in Distinct-1, and 1.3% in Distinct-2. These results demonstrate the effectiveness of our approach, which not only generates explanations closely aligned with references but also excels in improving diversity. Moreover, given the larger scale of the *Yelp* dataset, which better reflects real-world scenarios, these findings further validate the effectiveness of our method.

5.3 Ablation Study

In this section, we analyze the effect of different components in our framework. The variants are as follows: (1) The original model is the PETER model, which serves as the baseline. (2) The D4C (w/o Sem, w/o Adv) variant creates counterfactuals using the basic Abduction-Action-Prediction

Table 8. Performance Comparison of Variants of Our Framework on the *AM-CDs* Dataset

Methods	BLEU-3	BLEU-4	METEOR	DIST-1	DIST-2
AM-Electronics					
Original	1.77	0.99	8.41	0.78	2.64
D4C (w/o Sem, w/o Adv)	1.62	0.90	8.63	0.65	1.98
D4C (w/ Sem, w/o Adv)	1.85	1.15	9.12	0.73	2.30
D4C (w/o Sem, w/ Adv)	<u>2.05</u>	1.10	<u>9.28</u>	<u>0.82</u>	<u>2.75</u>
D4C	2.23	1.21	9.69	0.93	3.57
AM-Movies					
Original	1.77	0.99	8.41	0.78	2.64
D4C (w/o Sem, w/o Adv)	1.65	0.92	8.72	0.64	1.90
D4C (w/ Sem, w/o Adv)	1.92	1.20	9.34	0.71	2.35
D4C (w/o Sem, w/ Adv)	<u>2.11</u>	1.17	<u>9.48</u>	<u>0.86</u>	<u>2.81</u>
D4C	2.29	1.26	9.82	0.98	4.07

Bold values indicate the best performance.

process but does not include semantics (Equation (13)) or adversarial learning (Equation (14)) for alignment. (3) D4C (w/ Sem, w/o Adv) adds semantics to represent users and items but leaves out adversarial alignment. (4) D4C (w/o Sem, w/ Adv) skips semantics and uses only ID embeddings with adversarial learning. (5) The full model, D4C, combines both semantics and adversarial learning to achieve the best performance. We evaluate performance using three metrics: BLEU score, METEOR score, and DIST score, reflecting text matching, comprehensiveness, and diversity, respectively. The results on the AM-CDs dataset are shown in Table 8.

From Table 8, we observe that D4C (w/o Sem, w/o Adv) performs worse than the original model in BLEU and DIST scores, suggesting that using randomly augmented training samples can harm personalized explanation generation. Second, adversarial learning is essential for enhancing the quality of counterfactual samples, which in turn improves explanation quality. This is evident from the performance of D4C (w/o Sem, w/ Adv), which achieves notable gains in BLEU-3, BLEU-4, METEOR, and DIST scores. These results show that adversarial learning effectively aligns text representations across domains, reducing the model's burden of learning general language patterns and enabling greater focus on user-item interaction features. On the other hand, D4C (w/ Sem, w/o Adv) improves BLEU scores over the original model due to semantic representations but struggles to capture diversity, as shown by its lower DIST scores. We speculate this is because the generated counterfactuals remain too similar to the original explanations, with fewer domain-specific changes. This likely causes the model to focus more on mimicking the actual explanations, leading to improved text quality but reduced diversity. Finally, the patterns observed in the ablation study variants remain consistent when using *AM-Electronics* and *AM-Movies* as auxiliary datasets, demonstrating the robustness of our framework in improving both text quality and diversity across different auxiliary domain data.

5.4 Further Analysis

In this section, we analyze the effectiveness of our counterfactual framework to gain deeper insights of the approach. This includes hyper-parameter analysis across various datasets, visual analysis of enhanced causality, qualitative case studies of the generated counterfactual samples, and case studies of explanation generation for recommendations.

5.4.1 Analysis of Hyper-Parameter Effects. Our approach introduces a learning weight, η , to train the model on augmented counterfactual samples simultaneously, as described in Equation (18).

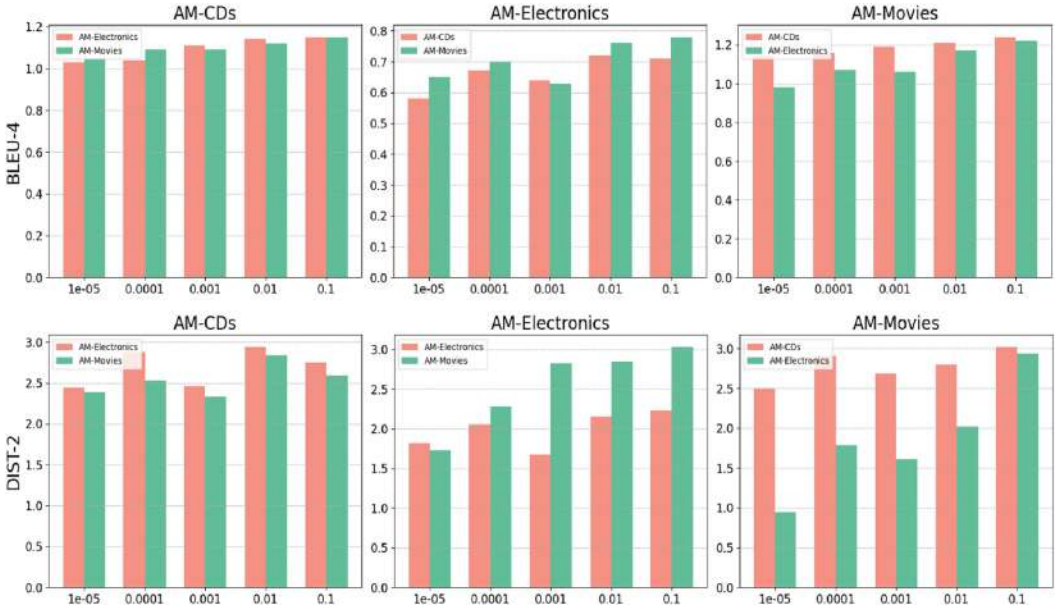


Fig. 5. Impact of the regularization weight η on BLEU-4 (top row) and DIST-2 (bottom row) for Amazon datasets.

To evaluate the effectiveness of this regularization term, we conduct experiments to analyze the impact of different η values. After generating the counterfactual dataset using our counterfactual reasoning method, we perform a grid search over learning rates $\{5e-4, 1e-4, 5e-4, 1e-3\}$ and η values from $\{1e-5, 1e-4, 1e-3, 1e-2, 1e-1\}$. For each η value, we average the results across all learning rates to reduce fluctuations and ensure a more reliable analysis of the regularization term influences. We evaluate performance using BLEU-4 and DIST-2 scores. BLEU-4 measures the quality of the explanation text, while DIST-2 evaluates the explanation diversity. Figure 5 shows the effect of the regularization weight η on BLEU-4 (top row) and DIST-2 (bottom row) scores for the Amazon datasets, while Figure 6 presents similar analyses for the TripAdvisor and Yelp datasets.

It can be observed that, as the regularization weight η increases, both BLEU-4 and DIST-2 scores improve consistently across all datasets, regardless of the auxiliary domain used. Specifically, the steady improvement in BLEU-4 indicates that higher values of η help the model generate more accurate and coherent explanations. In parallel, the significant gains in DIST-2 demonstrate that our approach effectively addresses the degeneration issue by reducing repetitive patterns and fostering more diverse explanations. These findings confirm the effectiveness of our theoretical approach to counterfactual data augmentation: with an appropriately tuned value of η , the model achieves optimal performance in both explanation accuracy and diversity.

5.4.2 Visual Analysis on Enhanced Causality. In Section 4, we use counterfactual reasoning to generate domain counterfactual samples. We argue that this approach effectively addresses degeneration by encouraging the model to learn enhanced interaction relationships for generating explanations, rather than simply relying on domain-specific text features. This is achieved through our proposed causality regularization mechanism. To validate this, we compared the contributions of user and item features in the generated tokens by analyzing the attention scores from Transformer layers in the D4C model and the PETER model. Specifically, we selected eight fixed batches from the test set, resulting in 1,024 samples (128 per batch). During the generation process, we recorded

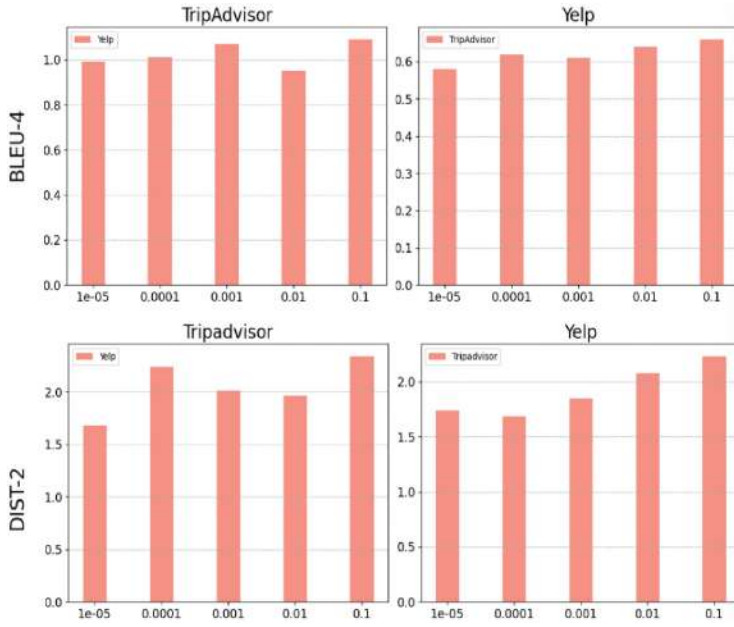


Fig. 6. Impact of the regularization weight η on BLEU-4 (top row) and DIST-2 (bottom row) for TripAdvisor and Yelp datasets.

the attention scores for these samples and calculated their average. The attention scores from the first and second layers of the generator models are shown in heatmaps, visualized in Figure 7. In the heatmap, the y -axis represents the target tokens (i.e., the generated explanation tokens), and the x -axis represents the source tokens, including the user, item, and other context tokens (e.g., domain features like $\langle bos \rangle$ and previously generated tokens). Both axes are capped at 25 because we set this as the maximum number of generated tokens. The value in each pixel indicates the contribution of each source token to the generation process, which ranges from 0 to 1, where 0 represents minimal attention and 1 represents maximum attention.

From the figure, it is evident that our model captures more significant contributions from user and item features when generating target tokens. Interestingly, we found that, in the first layer, both the PETER and D4C models rely more heavily on $\langle bos \rangle$ for generation. When in the second layer, both models shift their focus toward user and item features, with our method showing stronger attention to these key features. This enhanced attention pattern demonstrates the improved causality in our approach, confirming that it better leverages user–item interactions for explanation generation.

5.4.3 Qualitative Case Study on Counterfactuals. To better understand the effectiveness of the counterfactual reasoning approach in generating counterfactual training samples, we conducted a qualitative case study on the *AM-CDs* dataset. Specifically, we selected user-generated explanations from the target domain (*AM-CDs*) and analyzed the counterfactual explanations generated by our model using two auxiliary domains, *AM-Electronics* and *AM-Movies*. Table 9 presents the ground-truth explanations alongside the counterfactual explanations generated from the two auxiliary domains.

For each group, the users are consistent across domains (e.g., User 1 in the ground-truth explanations is the same User 1 in the generated counterfactuals). From the table, we observe that counterfactuals generated using *AM-Electronics* as the auxiliary domain adapt to emphasize product-related features, such as sound quality, price, and technical attributes. For example, User 1’s ground-truth

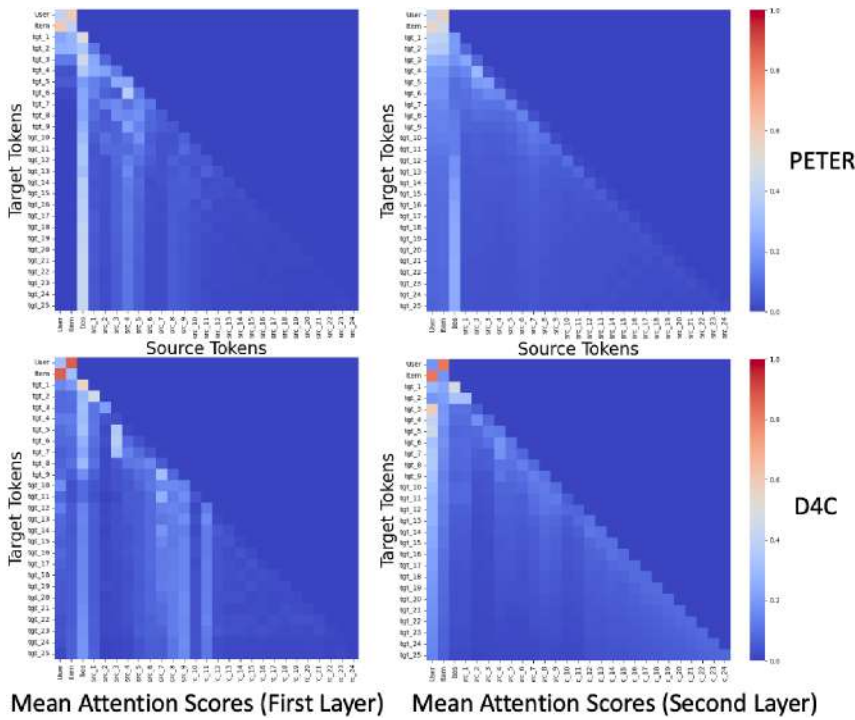


Fig. 7. Visual comparison of attention scores from the first and second layers of the D4C model and the PETER model.

Table 9. Examples of Ground-Truth Explanations from *AM-CDs* and the Corresponding Counterfactual Explanations Generated Using *AM-Electronics* and *AM-Movies* as Auxiliary Domains

	<i>Ground-Truth Explanations</i>
User 1	I love the guitar playing on the track “Beyond Here Lies Nothin”
User 2	The box features an actual reproduction 45 RPM single of that year’s smash single
User 3	The symphonic element added to the already extreme tunes makes this cd one great buy
User 4	Best rapper in the game since the best rapper retired
User 5	The music is solid and readily a good addition to any Heart fan
	<i>Generated Counterfactuals Using AM-Electronics as the Auxiliary Domain</i>
User 1	The first impressions are acoustic guitars on the same level
User 2	The extra features are greats and the moneys worth the price
User 3	I have a lot of research on the speakers and the same price
User 4	The video on the gamer is very good
User 5	The sound quality is great and the price is great for the product
	<i>Generated Counterfactuals Using AM-Movies as the Auxiliary Domain</i>
User 1	The DVD is a great addition to your collection and if you’re a big fan of the music
User 2	The special features are great and the DVD of the films and the a lot of fun
User 3	This album is a worthy addition to your Rush collection
User 4	The score is one of the best composers of the films ever made
User 5	I am a big fan of Heart and love their vocals and their music

Table 10. Explanations Generated by Different Methods on the *AM-CDs* Dataset (the First Group) and the *Yelp* Dataset (the Second Group)

Ground-truth	Going through the Tracks on the CD Makes You Realize that the Music on Offer Is Original and Superb
Dualpc	I'm a huge fan of the band
PETER	This is a great album to play of the same time
ReXPlug	This album is definitely worth owning a listen
ERRA	The best tracks on this collection are the ones that are not
PEFT	The title track is a great ending to a great album
PEPLER	And the guitars are great that are acoustic guitars on this album
AdaReX	This album is a great addition to the music collection and the crowd
D4C	The first two discs are the original tracks on the original version
Ground-Truth	The Wings Were Very Juicy and the Amount of Battery Coating Was Perfect—Saucy But Not Soggy
Dualpc	I've been here a few times and the food is good
PETER	The wings were delicious and the sushi was very fresh
ReXPlug	The food was good
ERRA	The food was good and the service was great
PEFT	The wings were good and the wings were good
PEPLER	The grilled chicken wings were delicious and then I had a nice flavor
AdaReX	The wings were good, the sauce was good, the wings were good a little spicy
D4C	The chicken is always tender & juicy crispy not dry over cooked

Keywords relevant to the actual ground-truth explanations are highlighted in **bold** within the sentences.

explanation, “I love the guitar playing on the track ‘Beyond Here Lies Nothin’,” reflects the user’s strong preference for musical elements, particularly guitar performance. When using *AM-Electronics* as the auxiliary domain, the counterfactual explanation shifts to, “The first impressions are acoustic guitars on the same level,” maintaining the user’s interest in guitar playing while also introducing product descriptions in electronics, such as “impressions” and “acoustic guitars,” reflecting how users might evaluate music equipment in electronics domain.

Similarly, for User 4, whose original explanation, “Best rapper in the game since the best rapper retired,” shows a clear interest in rap music and artists, the counterfactual generated using *AM-Movies* is transformed into “The score is one of the best composers of the films ever made.” While this shifts the focus from rap to movie scores, it maintains the user’s preference for evaluating the best in a given field (whether it be rappers or film composers), thereby integrating both the user’s interest and the domain-related features of the *AM-Movies* domain. Overall, this case study shows that the approach adjusts explanations to fit the characteristics of the auxiliary domain while preserving users’ original preferences to a large extent.

5.5 Case Study on Explanation Generation

We conduct a qualitative case study to evaluate explanations generated by different methods. Table 10 presents two examples: ground-truth explanations and corresponding outputs from various methods for the *AM-CDs* dataset (top) and the *Yelp* dataset (bottom). In the first example, single-domain methods like DualPC, PETER, ReXPlug, and ERRA produce generic explanations that fail to capture specific details of the CDs. For instance, ReXPlug provides a vague statement: “This album is definitely worth owning a listen.” In contrast, cross-domain methods such as PEFT, PEPLER, AdaReX, and D4C generate more detailed explanations. Examples include “acoustic guitars” from PEPLER, “music collection” from AdaReX, and “original tracks” from D4C (ours). These richer descriptions result in higher Distinct-1 and Distinct-2 scores, as shown in Table 3. Notably, D4C accurately highlights “original tracks,” aligning closely with the ground-truth attributes “original” and “superb.” This high level of words matching is reflected in superior ROUGE, BLEU, and METEOR scores in Table 3.

Similarly, in the second example from the *Yelp* dataset, the ground-truth explanation describes the wings as “very juicy” and “saucy but not soggy.” DualPC and ReXPlug provide generic responses, such as “the food is good,” which lack relevance to the actual description of chicken wings. Among the single-domain methods, PETER performs best, mentioning “wings were delicious” but erroneously includes “sushi.” Cross-domain methods perform better overall. PEFT mentions wings but is repetitive. AdaReX gives an adequate description of the wings and sauce but inaccurately adds spiciness, which is not in the ground-truth. D4C provides a highly detailed explanation: “The chicken is always tender and juicy, crispy but not dry or overcooked,” which closely aligns with the ground-truth and captures the user’s true preferences.

6 Conclusion

In this work, we addressed the degeneration in personalized explanations from a causal perspective. Through causal graph analysis, we found that domain-specific text attributes create shortcuts in model training. These shortcuts weaken the model’s ability to learn real user–item interactions for generating explanations. To address this, we not only provided theoretical insights for a solution but also proposed a novel framework, D4C, which employs counterfactual reasoning with adversarial learning. Experiments on real-world datasets demonstrated that our framework effectively mitigates degeneration by increasing the diversity of explanations while maintaining competitive explanation performance compared to state-of-the-art methods. We believe our counterfactual data augmentation, inspired by causal inference theories, goes beyond the current task. It has great potential to improve data quality in personalized tasks like conversational recommendations and to support the training of large-scale language models in recommender systems. We acknowledge that our approach depends on auxiliary domains that are different but related to the target domain. However, it is worth nothing that auxiliary data are relatively easy to acquire and does not necessarily need to align with the target domain type, such as explanatory data. As long as the auxiliary domain includes user reviews, which are commonly available on platforms like Amazon or other review-based systems, it can still support our framework. While our method currently focuses on two domains, it has the potential to extend beyond this setup. Future work could explore adapting the approach to handle multiple domains simultaneously, further broadening its applicability.

References

- [1] Amjad Almahairi, Kyle Kastner, Kyunghyun Cho, and Aaron Courville. 2015. Learning distributed representations from reviews for collaborative filtering. In *Proceedings of the 9th ACM Conference on Recommender Systems (RecSys ’15)*, 147–154.
- [2] Satandeep Banerjee and Alon Lavie. 2005. METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. In *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, 65–72.
- [3] Zefeng Cai and Zerui Cai. 2022. PEVAE: A hierarchical VAE for personalized explainable recommendation. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR ’22)*, 692–702.
- [4] Chong Chen, Min Zhang, Yiqun Liu, and Shaoping Ma. 2018. Neural attentional rating regression with review-level explanations. In *Proceedings of the 2018 World Wide Web Conference (WWW ’18)*, 1583–1592.
- [5] Xu Chen, Zhenlei Wang, Hongteng Xu, Jingsen Zhang, Yongfeng Zhang, Wayne Xin Zhao, and Ji-Rong Wen. 2022. Data augmented sequential recommendation based on counterfactual thinking. *IEEE Transactions on Knowledge and Data Engineering* 35, 9 (2022), 9181–9194.
- [6] Xu Chen, Yongfeng Zhang, and Zheng Qin. 2019. Dynamic explainable recommendation based on neural attentive models. In *Proceedings of the 33rd AAAI Conference on Artificial Intelligence (AAAI ’19)*, 53–60.
- [7] Hao Cheng, Shuo Wang, Wensheng Lu, Wei Zhang, Mingyang Zhou, Kezhong Lu, and Hao Liao. 2023. Explainable recommendation with personalized review retrieval and aspect learning. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL ’23)*, 51–64.

- [8] Zhiyong Cheng, Xiaojun Chang, Lei Zhu, Rose C. Kanjirathinkal, and Mohan Kankanhalli. 2019. MMALFM: Explainable recommendation by leveraging reviews and images. *ACM Transactions on Information Systems* 37, 2 (2019), 1–28.
- [9] Yoonhyuk Choi, Jiho Choi, Taewook Ko, Hyungho Byun, and Chong-Kwon Kim. 2022. Based domain disentanglement without duplicate users or contexts for cross-domain recommendation. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management (CIKM '22)*, 293–303.
- [10] Qiming Diao, Minghui Qiu, Chao-Yuan Wu, Alexander J. Smola, Jing Jiang, and Chong Wang. 2014. Jointly modeling aspects, ratings and sentiments for movie recommendation (JMARS). In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '14)*, 193–202.
- [11] Xin Dong, Jingchao Ni, Wei Cheng, Zhengzhang Chen, Bo Zong, Dongjin Song, Yanchi Liu, Haifeng Chen, and Gerard De Melo. 2020. Asymmetrical hierarchical networks with attentive interactions for interpretable review-based recommendation. In *Proceedings of the 34th AAAI Conference on Artificial Intelligence (AAAI '20)*, 7667–7674.
- [12] Chen Gao, Yu Zheng, Wenjie Wang, Fuli Feng, Xiangnan He, and Yong Li. 2024. Causal inference in recommender systems: A survey and future directions. *ACM Transactions on Information Systems* 42, 4 (2024), 1–32.
- [13] Shijie Geng, Zuohui Fu, Yingqiang Ge, Lei Li, Gerard De Melo, and Yongfeng Zhang. 2022. Improving personalized explanation generation through visualization. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL '22)*, 244–255.
- [14] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as language processing (RLP): A unified pretrain, personalized prompt and predict paradigm (P5). In *Proceedings of the 16th ACM Conference on Recommender Systems (RecSys '22)*, 299–315.
- [15] Xinyu Guan, Zhiyong Cheng, Xiangnan He, Yongfeng Zhang, Zhibo Zhu, Qinke Peng, and Tat-Seng Chua. 2019. Attentive aspect modeling for review-aware recommendation. *ACM Transactions on Information Systems* 37, 3 (2019), 1–27.
- [16] Deepesh V. Hada and Shirish K. Shevade. 2021. Rexplug: Explainable recommendation using plug-and-play language model. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '21)*, 81–91.
- [17] Ruining He and Julian McAuley. 2016. Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering. In *Proceedings of the 25th International Conference on World Wide Web (WWW '16)*, 507–517.
- [18] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural collaborative filtering. In *Proceedings of the 26th International Conference on World Wide Web (WWW '17)*, 173–182.
- [19] Jianchao Ji, Zelong Li, Shuyuan Xu, Max Xiong, Juntao Tan, Yingqiang Ge, Hao Wang, and Yongfeng Zhang. 2023. Counterfactual collaborative reasoning. In *Proceedings of the 16th ACM International Conference on Web Search and Data Mining (WSDM '23)*, 249–257.
- [20] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR '15)*, 1–15.
- [21] Taku Kudo and John Richardson. 2018. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP '18)*, 66–71.
- [22] Huayang Li, Tian Lan, Zihao Fu, Deng Cai, Lema Liu, Nigel Collier, Taro Watanabe, and Yixuan Su. 2023. Repetition in repetition out: Towards understanding neural text degeneration from the data perspective. In *Advances in Neural Information Processing Systems (NIPS '23)*, Vol. 36, 72888–72903.
- [23] Jiwei Li, Michel Galley, Chris Brockett, Jianfeng Gao, and William B. Dolan. 2016. A diversity-promoting objective function for neural conversation models. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics on Human Language Technologies (NAACL-HLT '16)*, 110–119.
- [24] Lei Li, Yongfeng Zhang, and Li Chen. 2020. Generate neural template explanations for recommendation. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management (CIKM '20)*, 755–764.
- [25] Lei Li, Yongfeng Zhang, and Li Chen. 2021. Personalized transformer for explainable recommendation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL-IJCNLP '21)*, 4947–4957.
- [26] Lei Li, Yongfeng Zhang, and Li Chen. 2023. Personalized prompt learning for explainable recommendation. *ACM Transactions on Information Systems* 41, 4 (2023), 1–26.
- [27] Piji Li, Zihao Wang, Zhaochun Ren, Lidong Bing, and Wai Lam. 2017. Neural rating regression with abstractive tips generation for recommendation. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '17)*, 345–354.
- [28] Qiming Li, Zhao Zhang, Fuzhen Zhuang, Yongjun Xu, and Chao Li. 2023. Topic-aware intention network for explainable recommendation with knowledge enhancement. *ACM Transactions on Information Systems* 41, 4 (2023), 1–23.
- [29] Chin-Yew Lin. 2004. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, 74–81.

- [30] Victoria Lin, Louis-Philippe Morency, Dimitrios Dimitriadis, and Srinagesh Sharma. 2023. Counterfactual augmentation for multimodal learning under presentation bias. In *Proceedings of the Findings of the Association for Computational Linguistics (EMNLP '23)*, 592–606.
- [31] Guang Ling, Michael R. Lyu, and Irwin King. 2014. Ratings meet reviews, a combined approach to recommend. In *Proceedings of the 8th ACM Conference on Recommender Systems (RecSys '14)*, 105–112.
- [32] Donghua Liu, Jing Li, Bo Du, Jun Chang, and Rong Gao. 2019. Daml: Dual attention mutual learning between ratings and reviews for item recommendation. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '19)*, 344–352.
- [33] Wei Liu, Jun Wang, Haozhao Wang, Ruixuan Li, Zhiying Deng, YuanKai Zhang, and Yang Qiu. 2023. D-separation for causal self-explanation. In *Advances in Neural Information Processing Systems (NIPS '23)*, Vol. 36, 43620–43633.
- [34] Hongyu Lu, Weizhi Ma, Yifan Wang, Min Zhang, Xiang Wang, Yiqun Liu, Tat-Seng Chua, and Shaoping Ma. 2023. User perception of recommendation explanation: Are your explanations what users need? *ACM Transactions on Information Systems* 41, 2 (2023), 1–31.
- [35] Julian McAuley and Jure Leskovec. 2013. Hidden factors and hidden topics: Understanding rating dimensions with review text. In *Proceedings of the 7th ACM Conference on Recommender Systems (RecSys '13)*, 165–172.
- [36] Julian John McAuley and Jure Leskovec. 2013. From amateurs to connoisseurs: Modeling the evolution of user expertise through online reviews. In *Proceedings of the 22nd International Conference on World Wide Web (WWW '13)*, 897–908.
- [37] Jianmo Ni, Jiacheng Li, and Julian McAuley. 2019. Justifying recommendations using distantly-labeled reviews and fine-grained aspects. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP '19)*, 188–197.
- [38] Sicheng Pan, Dongsheng Li, Hansu Gu, Tun Lu, Xufang Luo, and Ning Gu. 2022. Accurate and explainable recommendation via review rationalization. In *Proceedings of the ACM Web Conference 2022 (WWW '22)*, 3092–3101.
- [39] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL '02)*, 311–318.
- [40] Judea Pearl. 2009. Causal inference in statistics: An overview. *Statistics Surveys* 3 (2009), 96–146.
- [41] Judea Pearl. 2012. The do-calculus revisited. In *Proceedings of the 28th Conference on Uncertainty in Artificial Intelligence (UAI '12)*, 3–11.
- [42] Judea Pearl, Madelyn Glymour, and Nicholas P. Jewell. 2016. *Causal Inference in Statistics: A Primer*. John Wiley & Sons.
- [43] Yuta Saito and Thorsten Joachims. 2021. Counterfactual learning and evaluation for recommender systems: Foundations, implementations, and recent advances. In *Proceedings of the 15th ACM Conference on Recommender Systems (RecSys '21)*, 828–830.
- [44] Jie Shuai, Le Wu, Kun Zhang, Peijie Sun, Richang Hong, and Meng Wang. 2023. Topic-enhanced graph neural networks for extraction-based explainable recommendation. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '23)*, 1188–1197.
- [45] Peijie Sun, Le Wu, Kun Zhang, Yanjie Fu, Richang Hong, and Meng Wang. 2020. Dual learning for explainable recommendation: Towards unifying user preference prediction and review generation. In *Proceedings of the Web Conference 2020 (WWW '20)*, 837–847.
- [46] Peijie Sun, Le Wu, Kun Zhang, Yu Su, and Meng Wang. 2021. An unsupervised aspect-aware recommendation model with explanation text generation. *ACM Transactions on Information Systems* 40, 3 (2021), 1–29.
- [47] Juntao Tan, Shuyuan Xu, Yingqiang Ge, Yunqi Li, Xu Chen, and Yongfeng Zhang. 2021. Counterfactual explainable recommendation. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management (CIKM '21)*, 1784–1793.
- [48] Yunzhi Tan, Min Zhang, Yiqun Liu, and Shaoping Ma. 2016. Rating-boosted latent topics: Understanding users and items with ratings and reviews. In *Proceedings of the 25th International Joint Conference on Artificial Intelligence (IJCAI '16)*, 2640–2646.
- [49] Yi Tay, Anh Tuan Luu, and Siu Cheung Hui. 2018. Multi-pointer co-attention networks for recommendation. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '18)*, 2309–2318.
- [50] Nava Tintarev and Judith Masthoff. 2007. A survey of explanations in recommender systems. In *Proceedings of the 2007 IEEE 23rd International Conference on Data Engineering Workshop (ICDEW '07)*. IEEE, 801–810.
- [51] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems (NIPS '17)*, 5998–6008.
- [52] Sandra Wachter, Brent Mittelstadt, and Chris Russell. 2017. Counterfactual explanations without opening the black box: Automated decisions and the GDPR. *Harvard Journal of Law and Technology* 31 (2017), 841.

- [53] Cheng Wang, Mathias Niepert, and Hui Li. 2019. Recsys-Dan: Discriminative adversarial networks for cross-domain recommender systems. *IEEE Transactions on Neural Networks and Learning Systems* 31, 8 (2019), 2731–2740.
- [54] Jun Wang, Haoxuan Li, Chi Zhang, Dongxu Liang, Enyun Yu, Wenwu Ou, and Wenjia Wang. 2023. Counterclr: Counterfactual contrastive learning with non-random missing data in recommendation. In *Proceedings of the 2023 IEEE International Conference on Data Mining (ICDM '23)*, 1355–1360.
- [55] Nan Wang, Hongning Wang, Yiling Jia, and Yue Yin. 2018. Explainable recommendation via multi-task learning in opinionated text data. In *Proceedings of the 41st International ACM SIGIR Conference on Research & Development in Information Retrieval (SIGIR '18)*, 165–174.
- [56] Wenjie Wang, Fuli Feng, Xiangnan He, Hanwang Zhang, and Tat-Seng Chua. 2021. Clicks can be cheating: Counterfactual recommendation for mitigating clickbait issue. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '21)*, 1288–1297.
- [57] Wenjie Wang, Yang Zhang, Haoxuan Li, Peng Wu, Fuli Feng, and Xiangnan He. 2023. Causal recommendation: Progresses and future directions. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '23)*, 3432–3435.
- [58] Xiaolei Wang, Kun Zhou, Xinyu Tang, Wayne Xin Zhao, Fan Pan, Zhao Cao, and Ji-Rong Wen. 2023. Improving conversational recommendation systems via counterfactual data simulation. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '23)*, 2398–2408.
- [59] Zhenlei Wang, Jingsen Zhang, Hongteng Xu, Xu Chen, Yongfeng Zhang, Wayne Xin Zhao, and Ji-Rong Wen. 2021. Counterfactual data-augmented sequential recommendation. In *Proceedings of the 44th international ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '21)*, 347–356.
- [60] Tianxin Wei, Fuli Feng, Jiawei Chen, Ziwei Wu, Jinfeng Yi, and Xiangnan He. 2021. Model-agnostic counterfactual reasoning for eliminating popularity bias in recommender system. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining (KDD '21)*, 1791–1800.
- [61] Yao Wu and Martin Ester. 2015. Flame: A probabilistic model combining aspect based opinion mining and collaborative filtering. In *Proceedings of the 8th ACM International Conference on Web Search and Data Mining (WSDM '15)*, 199–208.
- [62] Kun Xiong, Wenwen Ye, Xu Chen, Yongfeng Zhang, Wayne Xin Zhao, Binbin Hu, Zhiqiang Zhang, and Jun Zhou. 2021. Counterfactual review-based recommendation. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management (CIKM '21)*, 2231–2240.
- [63] Jin Xu, Xiaojiang Liu, Jianhao Yan, Deng Cai, Huayang Li, and Jian Li. 2022. Learning to break the loop: Analyzing and mitigating repetitions for neural text generation. In *Advances in Neural Information Processing Systems (NIPS '22)*, Vol. 35, 3082–3095.
- [64] Shuyuan Xu, Yingqiang Ge, Yunqi Li, Zuohui Fu, Xu Chen, and Yongfeng Zhang. 2023. Causal collaborative filtering. In *Proceedings of the 2023 ACM SIGIR International Conference on Theory of Information Retrieval (ICTIR '23)*, 235–245.
- [65] Shuyuan Xu, Jianchao Ji, Yunqi Li, Yingqiang Ge, Juntao Tan, and Yongfeng Zhang. 2023. Causal inference for recommendation: Foundations, methods and applications. arXiv:2301.04016. Retrieved from <https://arxiv.org/abs/2301.04016>
- [66] Shuyuan Xu, Yunqi Li, Shuchang Liu, Zuohui Fu, Yingqiang Ge, Xu Chen, and Yongfeng Zhang. 2021. Learning causal explanations for recommendation. In *Proceedings of the 1st International Workshop on Causality in Search and Recommendation*, Vol. 2911, 13–25.
- [67] Shuyuan Xu, Juntao Tan, Zuohui Fu, Jianchao Ji, Shelby Heinecke, and Yongfeng Zhang. 2022. Dynamic causal collaborative filtering. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management (CIKM '22)*, 2301–2310.
- [68] Shuyuan Xu, Juntao Tan, Shelby Heinecke, Vena Jia Li, and Yongfeng Zhang. 2023. Deconfounded causal collaborative filtering. *ACM Transactions on Recommender Systems* 1, 4 (2023), 1–25.
- [69] Shuyuan Xu, Da Xu, Evren Korpeoglu, Sushant Kumar, Stephen Guo, Kannan Achan, and Yongfeng Zhang. 2024. Causal structure learning for recommender system. *ACM Transactions on Recommender Systems* 3, 1 (2024), 1–23.
- [70] Zhichao Xu, Hansi Zeng, Juntao Tan, Zuohui Fu, Yongfeng Zhang, and Qingyao Ai. 2023. A reusable model-agnostic framework for faithfully explainable recommendation and system scrutability. *ACM Transactions on Information Systems* 42, 1 (2023), 1–29.
- [71] Aobo Yang, Nan Wang, Hongbo Deng, and Hongning Wang. 2021. Explanation as a defense of recommendation. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining (WSDM '21)*, 1029–1037.
- [72] Dianer Yu, Qian Li, Xiangmeng Wang, Qing Li, and Guandong Xu. 2023. Counterfactual explainable conversational recommendation. *IEEE Transactions on Knowledge and Data Engineering* 33, 6 (2023), 2388–2400.
- [73] Mo Yu, Shiyu Chang, Yang Zhang, and Tommi S. Jaakkola. 2019. Rethinking cooperative rationalization: Introspective extraction and complement control. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and International Joint Conference on Natural Language Processing (EMNLP '19)*, 4094–4103.
- [74] Yi Yu, Kazunari Sugiyama, and Adam Jatowt. 2023. AdaReX: Cross-domain, adaptive, and explainable recommender system. In *Proceedings of the 1st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region (SIGIR-AP '23)*, 272–281.

- [75] Ruohan Zhan, Changhua Pei, Qiang Su, Jianfeng Wen, Xueliang Wang, Guanyu Mu, Dong Zheng, Peng Jiang, and Kun Gai. 2022. Deconfounding duration bias in watch-time prediction for video recommendation. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '22)*, 4472–4481.
- [76] Yongfeng Zhang and Xu Chen. 2020. Explainable recommendation: A survey and new perspectives. arXiv:1804.11192. Retrieved from <https://arxiv.org/abs/1804.11192>
- [77] Yang Zhang, Fuli Feng, Xiangnan He, Tianxin Wei, Chonggang Song, Guohui Ling, and Yongdong Zhang. 2021. Causal intervention for leveraging popularity bias in recommendation. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '21)*, 11–20.
- [78] Yongfeng Zhang, Guokun Lai, Min Zhang, Yi Zhang, Yiqun Liu, and Shaoping Ma. 2014. Explicit factor models for explainable recommendation based on phrase-level sentiment analysis. In *Proceedings of the 37th International ACM SIGIR Conference on Research & Development in Information Retrieval (SIGIR '14)*, 83–92.
- [79] Yu Zheng, Chen Gao, Xiang Li, Xiangnan He, Yong Li, and Depeng Jin. 2021. Disentangling user interest and conformity for recommendation with causal embedding. In *Proceedings of the Web Conference 2021 (WWW '21)*, 2980–2991.

Received 14 May 2024; revised 12 December 2024; accepted 16 December 2024